

Chapter 2: Memory Hierarchy Design

Introduction (Section 2.1, Appendix B)

Caches

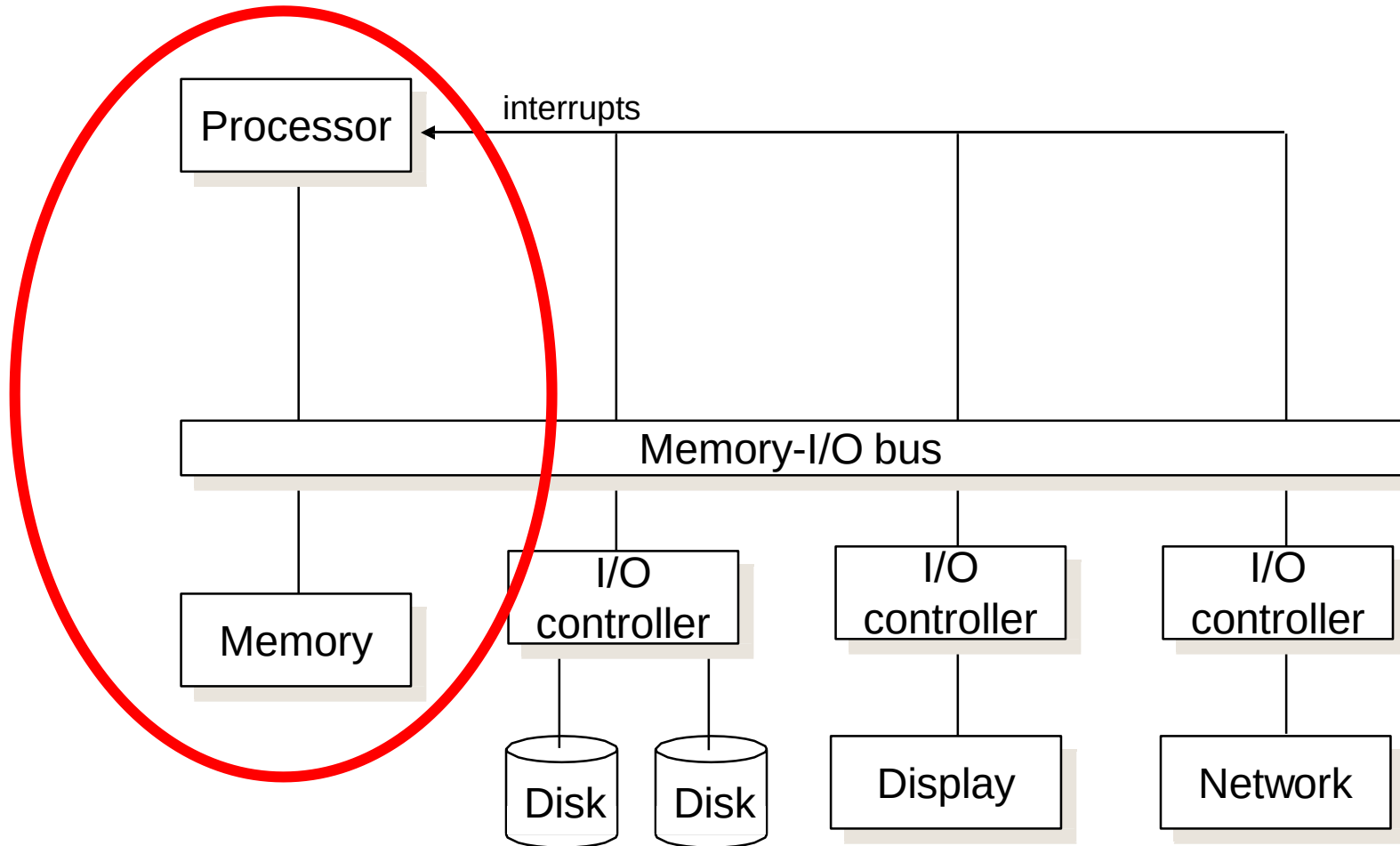
- Review of basics (Section 2.1, Appendix B)

- Advanced methods

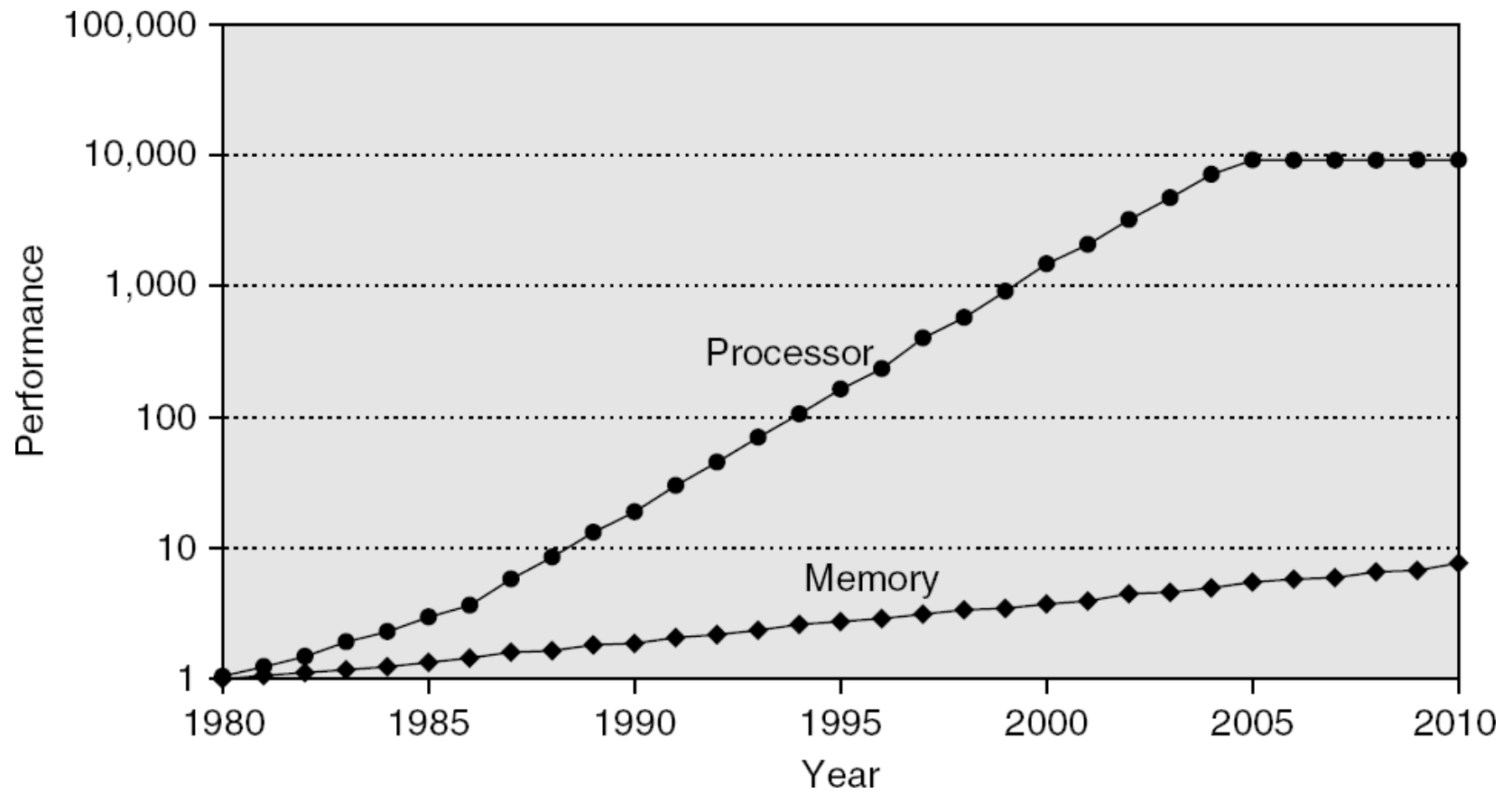
Main Memory

Virtual Memory

Early computer system



Recall: Processor-memory gap



Memory technology trends

Memory: Dynamic random access memory DRAM		
Year	Capacity	Latency
1980	64 kbit	250 nsec
1983	256 kb	220 ns
1986	1 Mb	190 ns
1989	4 Mb	165 ns
1992	16 Mb	145 ns
1995	64 Mb	120 ns
1999	128 Mb	110 ns
2003	512 Mb	90 ns
2007	2 Gb	60 ns

Capacity

DRAM: 4x in 3 years

Latency

DRAM: 2x in 10 years

Ideal memory

We **want** a large, fast memory

...But technology doesn't let us have this!

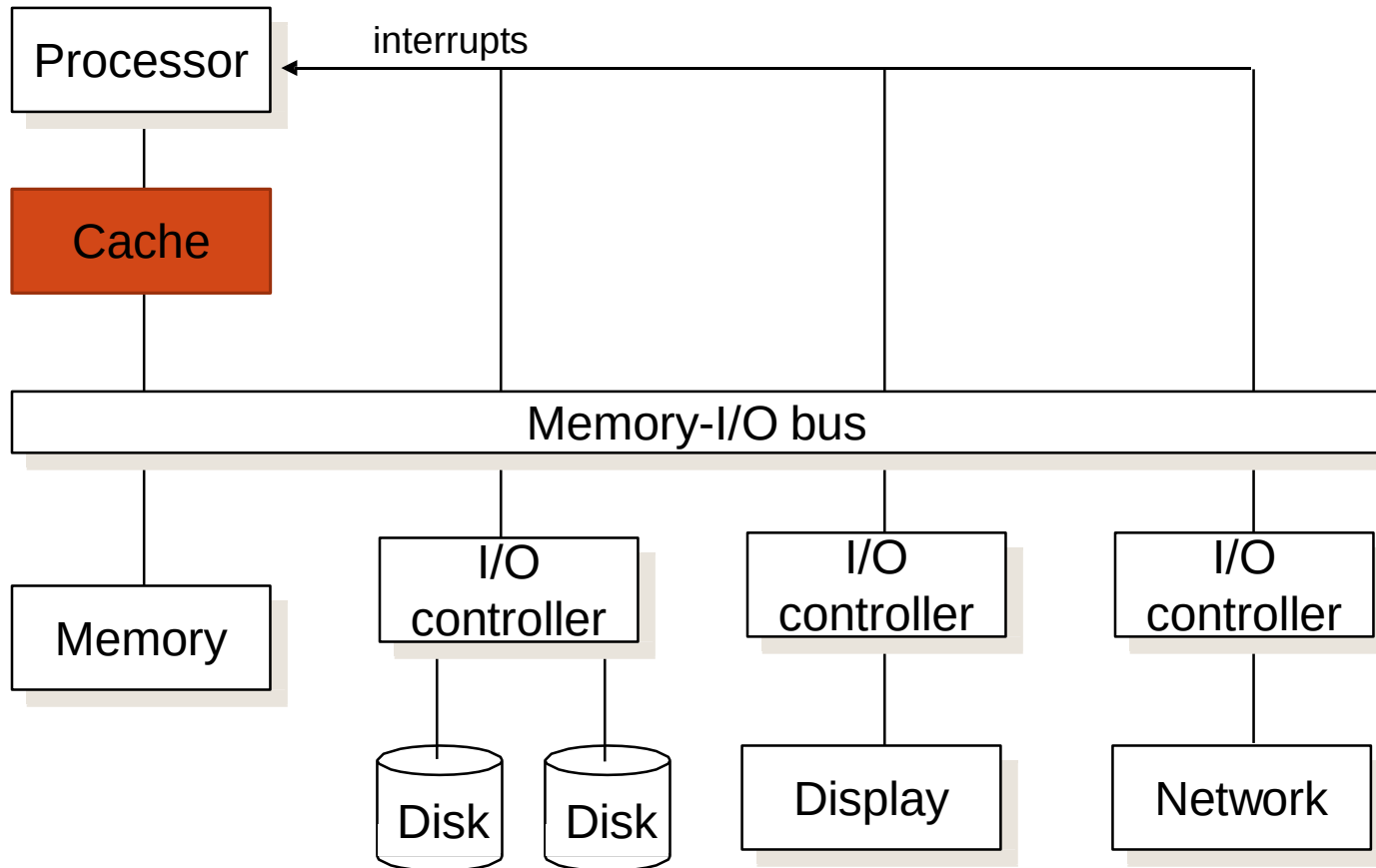
Key observation: **Locality**

All data are not equal

Some data are accessed more often than others

Architects solution: **Caches** Memory hierarchy

Modern computer system



Memory Hierarchies: Key Principles

Make the common case fast

Common → Principle of locality

Fast → Smaller is faster, but more expensive

Principle of Locality

Temporal locality

Spatial locality

Examples:

*Principle of Locality***

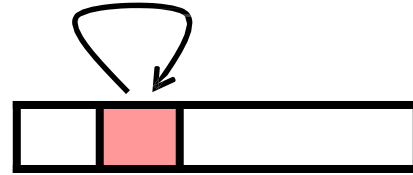
Temporal locality

Locality in time

If a datum has been recently referenced, it is likely to be referenced again

Spatial locality

Examples:

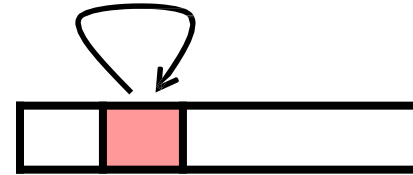


*Principle of Locality***

Temporal locality

Locality in time

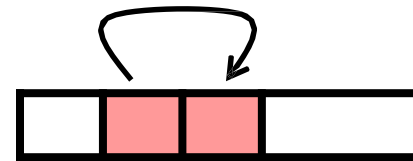
If a datum has been recently referenced, it is likely to be referenced again



Spatial locality

Locality in space

When a datum is referenced, neighboring data are likely to be referenced soon



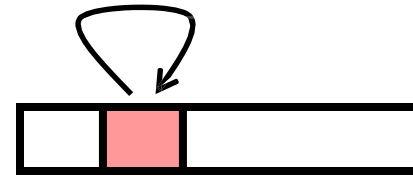
Examples:

*Principle of Locality***

Temporal locality

Locality in time

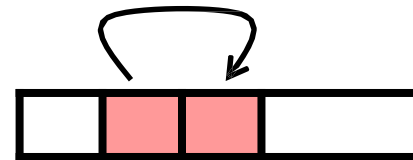
If a datum has been recently referenced, it is likely to be referenced again



Spatial locality

Locality in space

When a datum is referenced, neighboring data are likely to be referenced soon



Examples:

Temporal locality: Code in a loop

Spatial locality: Top of stack, Sequential instructions, Structure references

Principle of Locality: Example

Examples:

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += a[i];  
return sum;
```

Data access

Access contiguous array elements (step: 1) **Spatial locality**

Access i, sum in each loop **Temporal locality**

Instruction access

Sequential read instruction **Spatial locality**

Repeat the instructions in the loop **Temporal locality**

Memory Types

Volatile Memories

- When the power is interrupted, the stored data is quickly lost.
- Static Random Access Memory, SRAM
- Dynamic Random Access Memory, DRAM

Non-Volatile Memories

- Can retain stored information even after power is removed;
- Hard disk drives, NAND flash, SSD, optical discs

Smaller is Faster

Registers are fastest memory

Smallest and most expensive

Static RAMs are faster than DRAMs

10X faster

10X less dense

DRAMs are faster than disk, flash

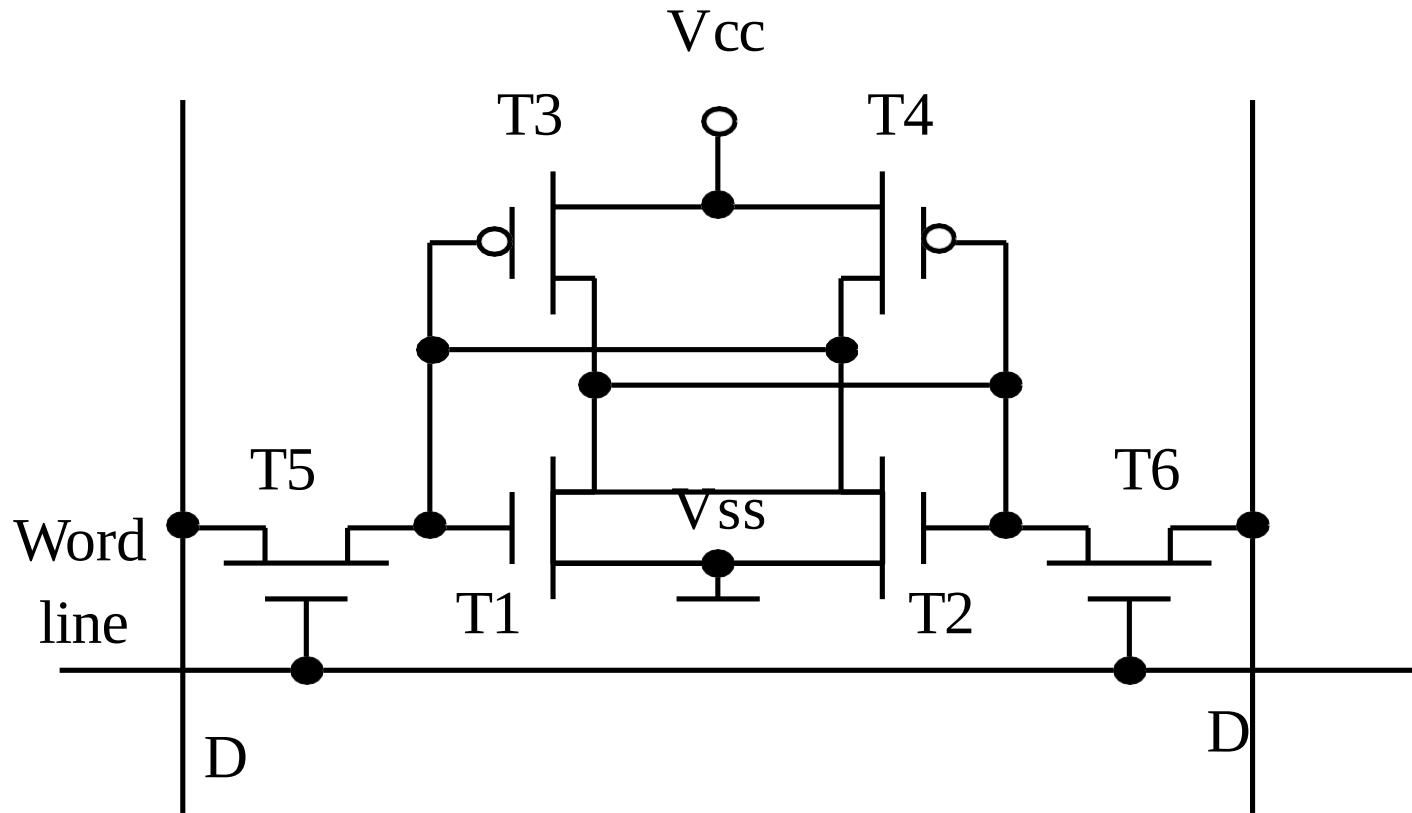
SRAM

SRAM: Static Random Access Memory

Static High cost, high power

10X faster than DRAM

10X less dense than DRAM



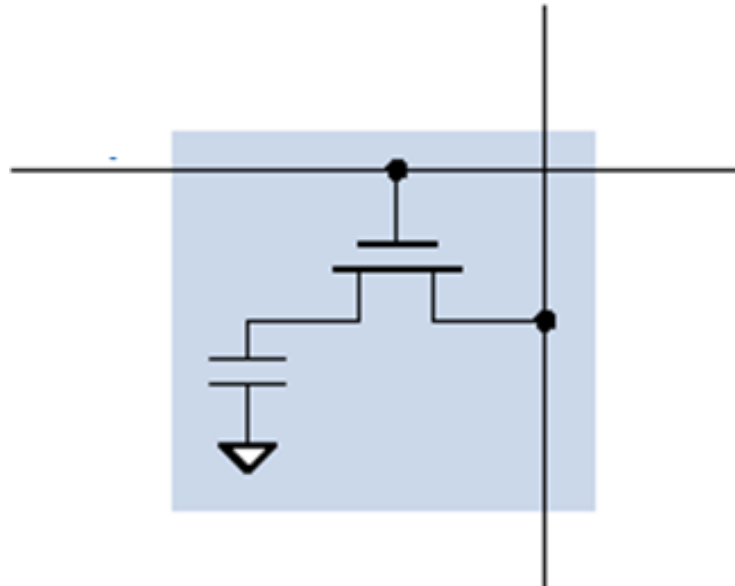
DRAM

DRAM: Dynamic Random Access Memory

Dynamic Less cost, less power

10X slower than SRAM

10X higher dense than SRAM



SRAM vs DRAM

Q: When does an architect use DRAM? SRAM?

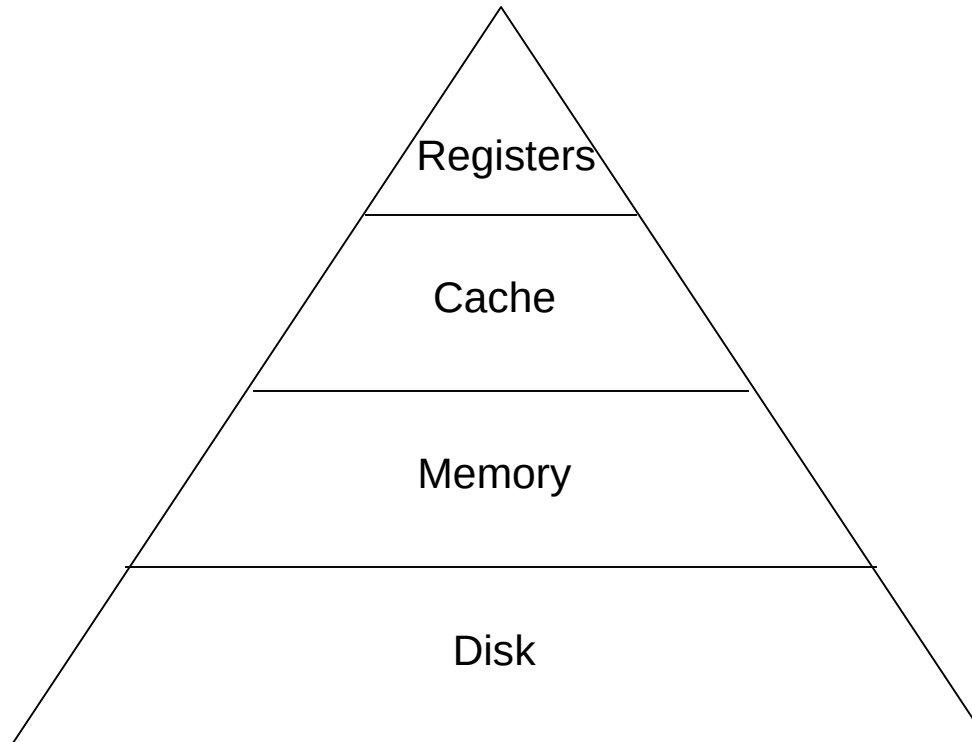
SRAM used for on-chip caches, register file

DRAM used for main memory

- Often with a different manufacturing process, optimized for density not speed
- That's why single chips with main memory + logic are rare
- “3D stacking” is changing this (kind of)

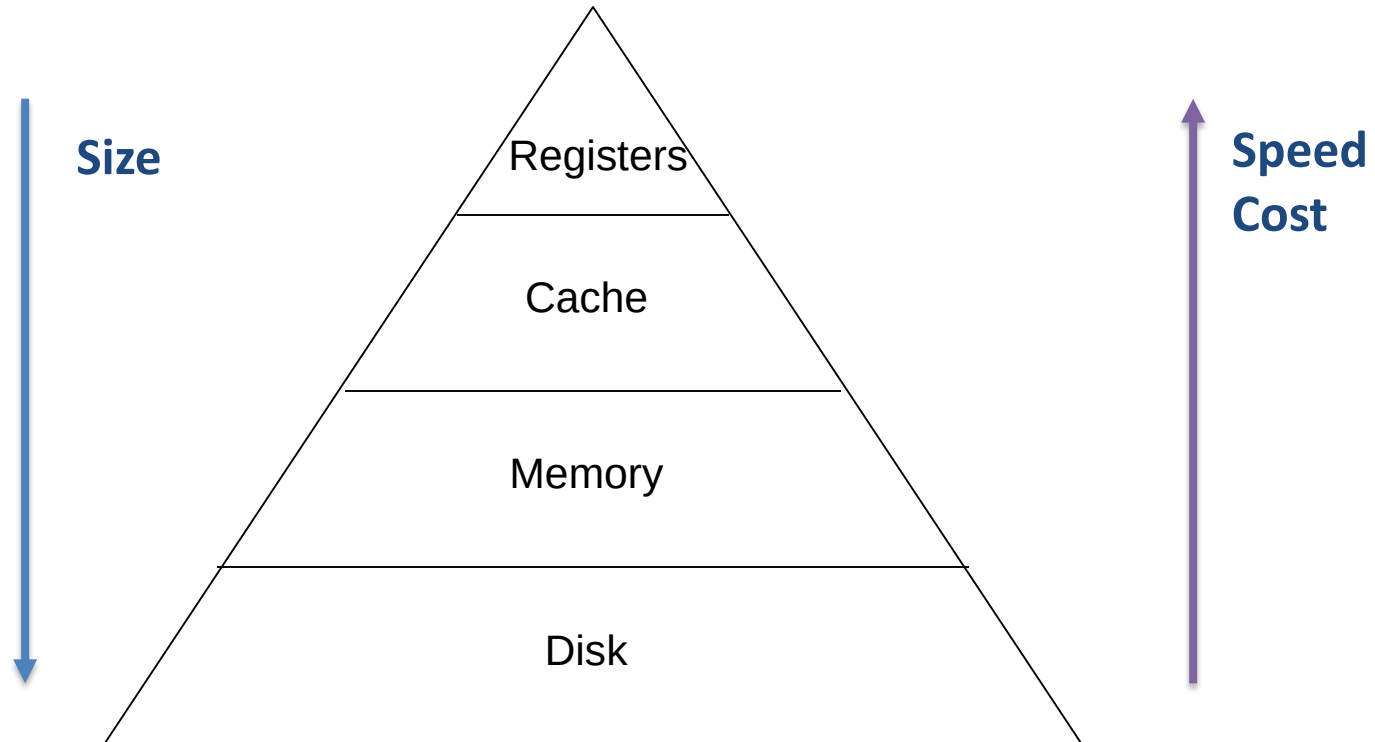
	SRAM	DRAM
Simplicity	✓	
Latency	✓	
Bandwidth	✓	
Density		✓

Memory Hierarchy



Type	Size	Speed (x proc. clk)
Registers		
Cache		
Memory		
Disk, Flash		

Memory Hierarchy**



Type	Size	Speed (x proc. clk)
Registers	32 to 128 I and F	1X
Cache	10s of KB to 10s of MB	~1 to 10X on-chip, ~10X off-chip
Memory	GB	~100X
Disk, Flash	GB to TB to ...	~1000000X

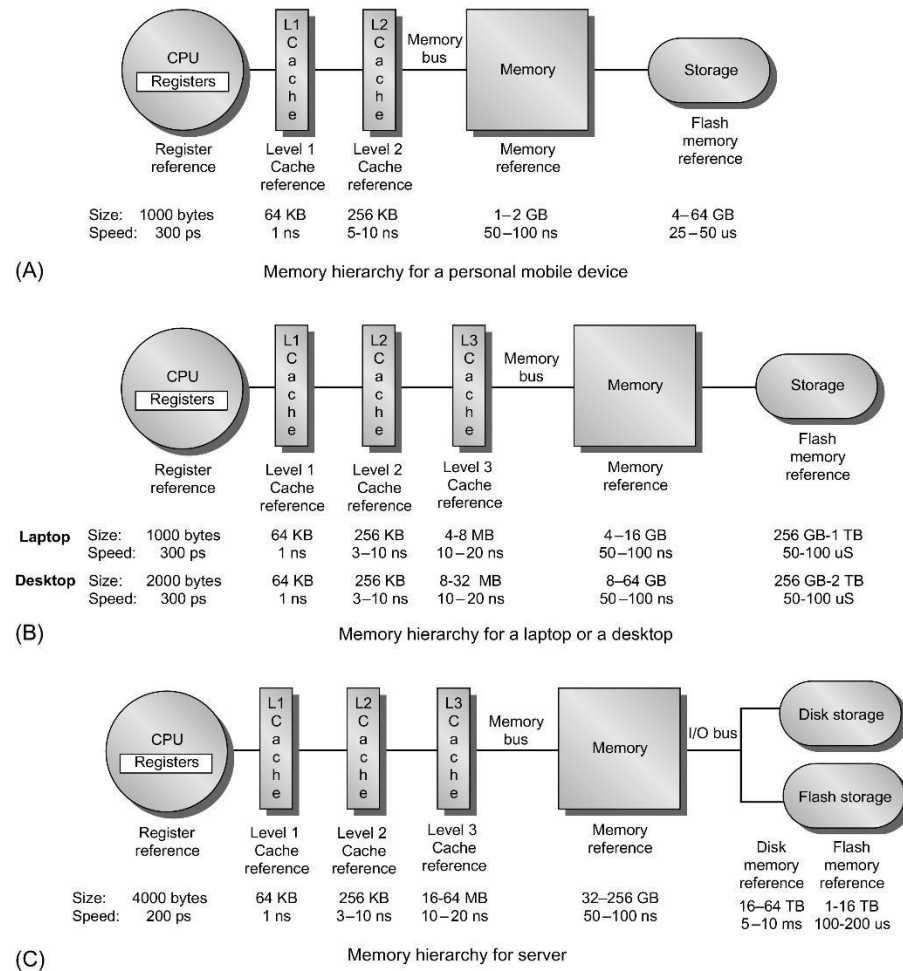


Figure 2.1 The levels in a typical memory hierarchy in a personal mobile device (PMD), such as a cell phone or tablet (A), in a laptop or desktop computer (B), and in a server (C). As we move farther away from the processor, the memory in the level below becomes slower and larger. Note that the time units change by a factor of 10^9 from picoseconds to milliseconds in the case of magnetic disks and that the size units change by a factor of 10^{10} from thousands of bytes to tens of terabytes. If we were to add warehouse-sized computers, as opposed to just servers, the capacity scale would increase by three to six orders of magnitude. Solid-state drives (SSDs) composed of Flash are used exclusively in PMDs, and heavily in both laptops and desktops. In many desktops, the primary storage system is SSD, and expansion disks are primarily hard disk drives (HDDs). Likewise, many servers mix SSDs and HDDs.

Memory Hierarchy Terminology

Block

Minimum unit that may be present

Usually fixed length

Hit – Block is found in upper level

Miss – Not found in upper level

Miss ratio – Fraction of references that miss

Hit Time – Time to access the upper level

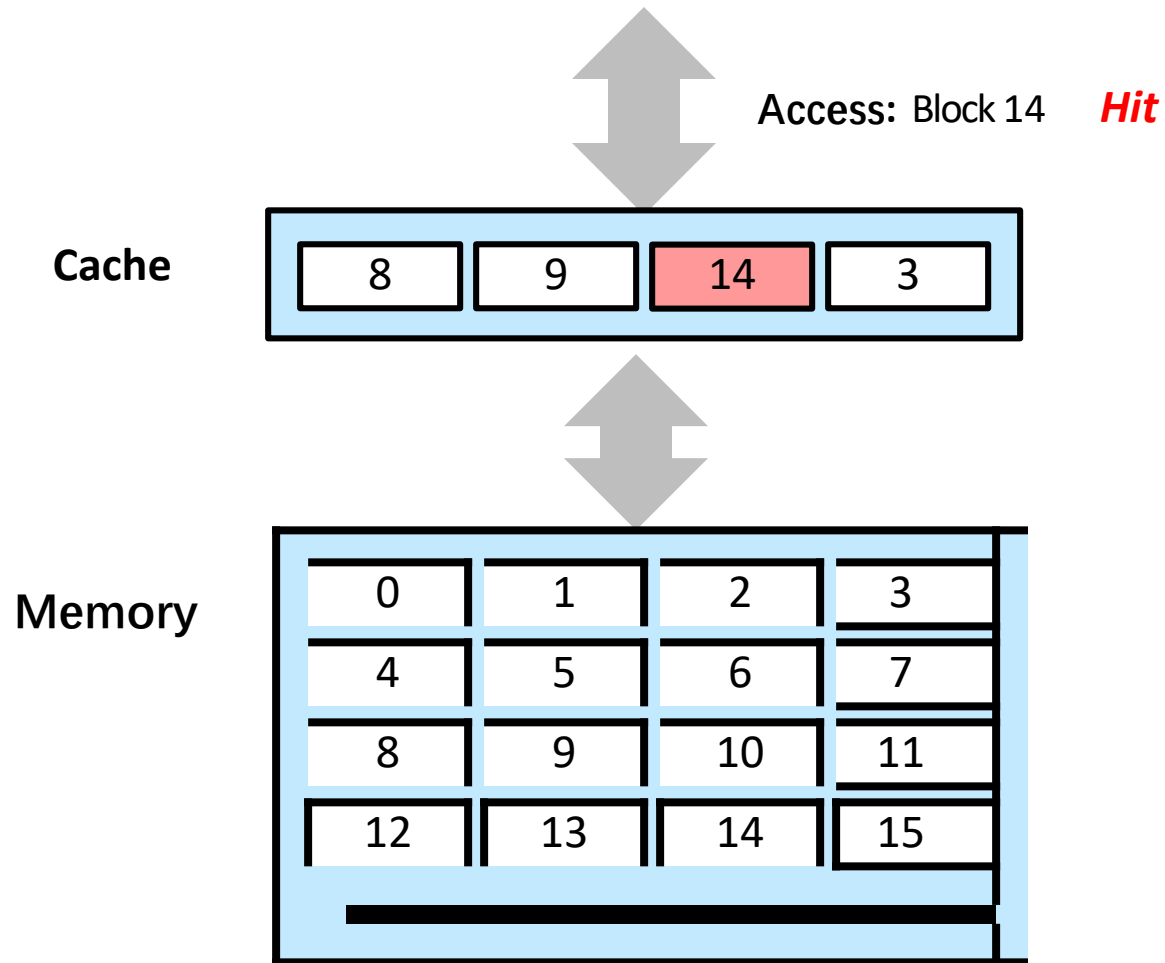
Miss Penalty

Time to replace block in upper level, plus the time to deliver the block to the CPU

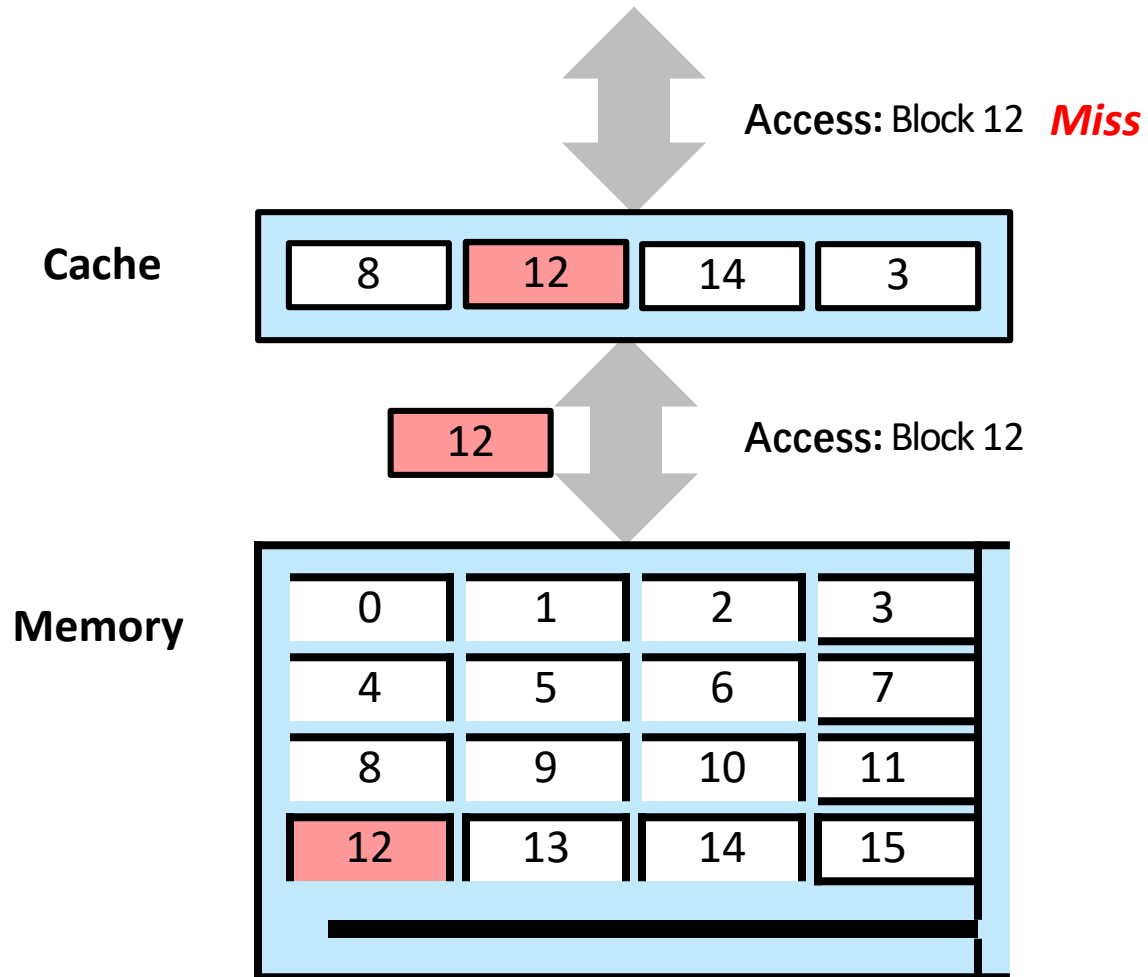
Access time – Time to get first word

Transfer time – Time for remaining words

Hit



Miss



Memory Hierarchy Terminology

Memory Address

Block•frame address	Offset
0101010101010101011	01010101

Block Names

Cache: Line

VM: Page

Memory Hierarchy Performance

Indirect measures of time can be misleading

MIPS can be misleading

So can Miss ratio

Average (effective) access time is better

$$t_{avg} =$$

Example:

$$t_{hit} = 1$$

$$t_{miss} = 20$$

$$\text{miss ratio} = .05$$

$$t_{avg} =$$

Effective access time is still an indirect measure

Memory Hierarchy Performance**

Time is always the ultimate measure

Indirect measures can be misleading

MIPS can be misleading

So can Miss ratio

Average (effective) access time is better

$$\begin{aligned} t_{avg} &= t_{hit} + miss\ ratio \times t_{miss} \\ &= t_{cache} + miss\ ratio \times t_{memory} \end{aligned}$$

Example:

$$t_{hit} = 1$$

$$t_{miss} = 20$$

$$miss\ ratio = .05$$

$$t_{avg} =$$

Effective access time is still an indirect measure

Memory Hierarchy Performance**

Time is always the ultimate measure

Indirect measures can be misleading

MIPS can be misleading

So can Miss ratio

Average (effective) access time is better

$$\begin{aligned}t_{avg} &= t_{hit} + miss\ ratio \times t_{miss} \\ &= t_{cache} + miss\ ratio \times t_{memory}\end{aligned}$$

Example:

$$t_{hit} = 1$$

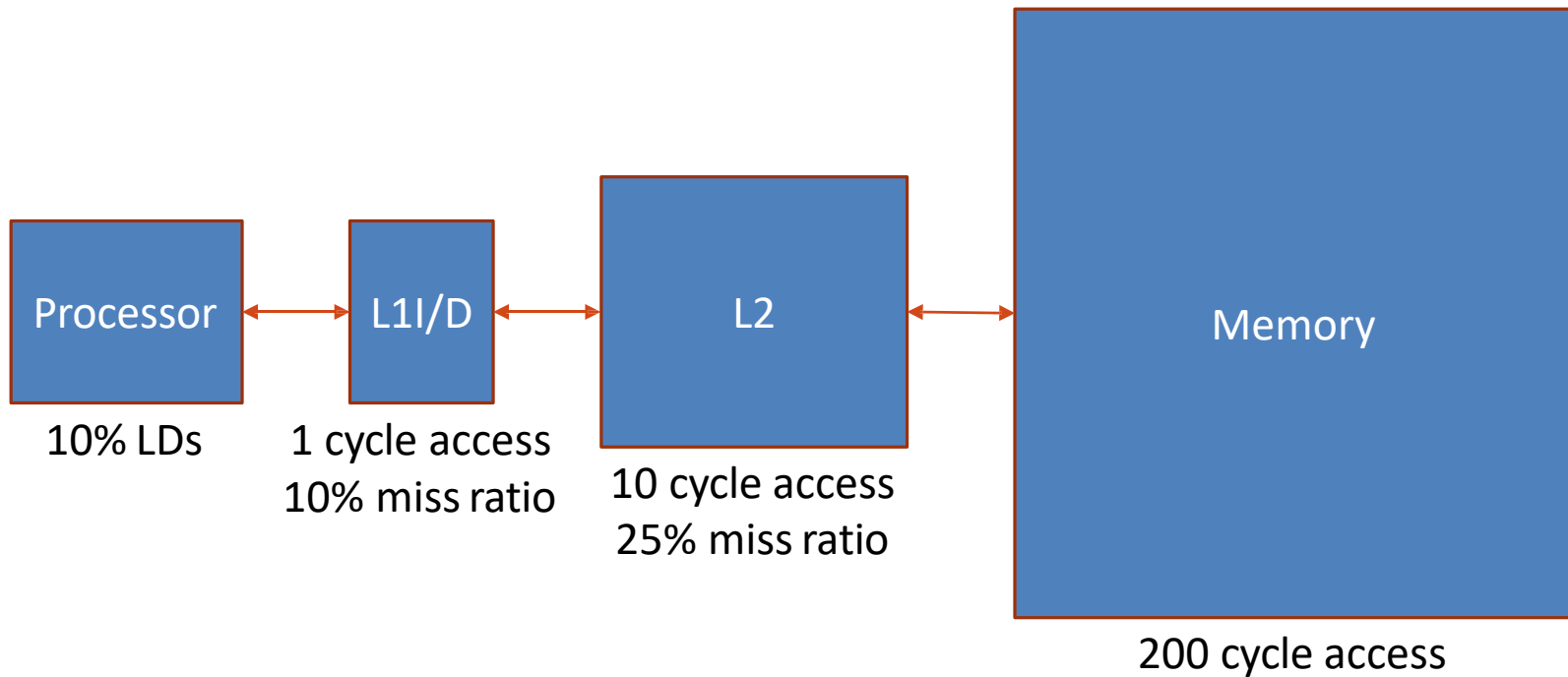
$$t_{miss} = 20$$

$$miss\ ratio = .05$$

$$t_{avg} = 1 + .05 \times 20 = 2$$

Effective access time is still an indirect measure

Example



Memory $t_{avg} = 200$ cycles

L2 $t_{avg} = 10 \text{ cycles} + 0.25 * 200 = 60$ cycles

L1 $t_{avg} = 1 \text{ cycle} + 0.10 * 60 \text{ cycles} = 7$ cycles

Memory CPI = $(1 + 0.10) * 7 = 7.7$ cycles

Example

Poor question:

Q: What is a reasonable miss ratio?

A: 1%, 2%, 5%, 10%, 20% ???

A better question

Q: What is a reasonable t_{avg} ?

(assume $t_{cache} = 1$ cycle, $t_{memory} = 20$ cycles)

A: 1.2, 1.5, 2.0 cycles

What's a reasonable t_{avg} ?

Example**

Poor question:

Q: What is a reasonable miss ratio?

A: 1%, 2%, 5%, 10%, 20% ???

A better question

Q: What is a reasonable t_{avg} ?

(assume $t_{cache} = 1$ cycle, $t_{memory} = 20$ cycles)

A: 1.2, 1.5, 2.0 cycles

What's a reasonable t_{avg} ?

Depends upon base CPI

$t_{avg} = 2.0$ might be OK for base $CPI = 10$,

but terrible for base $CPI = 1.2$

Example, cont.

Rearranging terms in

$$t_{avg} = t_{cache} + miss\ ratio \times t_{memory}$$

to solve for miss ratios yields

$$miss = \frac{(t_{avg} - t_{cache})}{t_{memory}}$$

Reasonable miss ratios (percent) - assume $t_{cache} = 1$

t_{memory} (cycles)	t_{avg} (cycles)		
	1.2	1.5	2.0
2	10.0	25.0	50.0
20	1.0	2.5	5.0
200	0.1	0.25	0.5

Proportional to acceptable t_{avg} degradation

Inversely proportional to t_{memory}

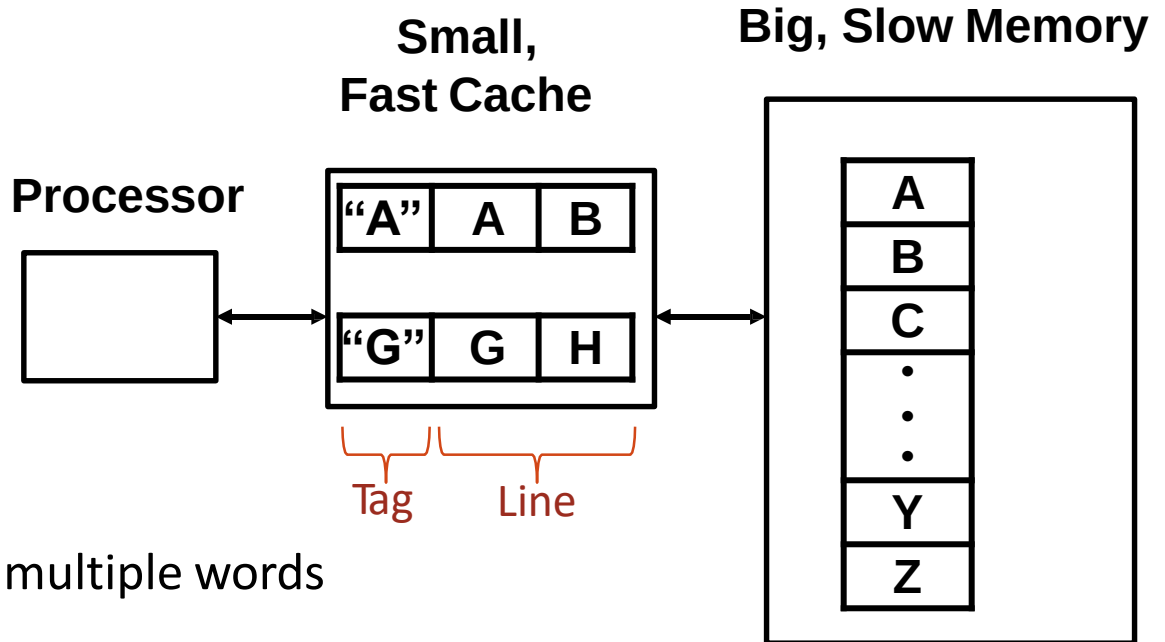
Caching: The basic idea

Main Memory

- Stores words
- A–Z in example

Cache

- Stores subset of next level
- E.g., ABGH in example
- An **inclusive** hierarchy
- **Tags** track what's in the cache
- Organized in **lines/blocks** of multiple words
- Exploit spatial locality
- Amortize overheads



Access

- Processor requests address from cache, which handles misses **itself**
- *What happens when processor accesses A? B? C?*

Basic Cache Questions

Block placement

Where can a block be placed in the cache?

Block Identification

How is a block found in the cache?

Block replacement

Which block should be replaced on a miss?

Write strategy

What happens on a write?

Cache Type

What type of information is stored in the cache?

Block Placement

FullyAssociative

Block goes in any block frame

Directmapped

Block goes in exactly one block frame

$(\text{Block frame \#}) \bmod (\# \text{ of blocks})$

SetAssociative

Block goes in exactly one set

$(\text{Block frame \#}) \bmod (\# \text{ of sets})$

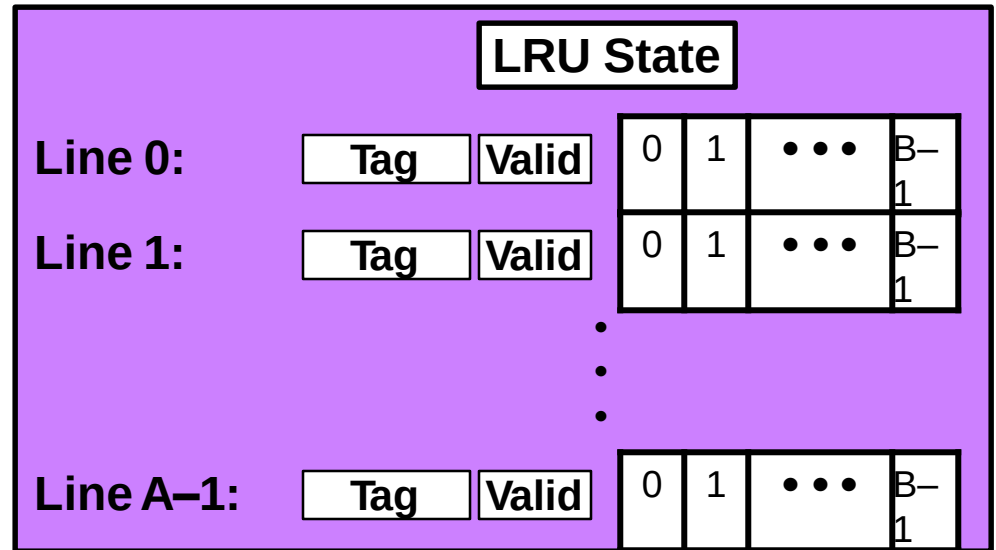
Example: Consider cache with 8 blocks, where does block 12 go?

Fully Associative Cache

Parameters

- Block = $B = 2^b$ bytes
- Cache size $C = 2^c$
- Cache consists of single **set** holding $A = C/B = 2^{c-b}$ lines

Entire Cache



Common in software caches

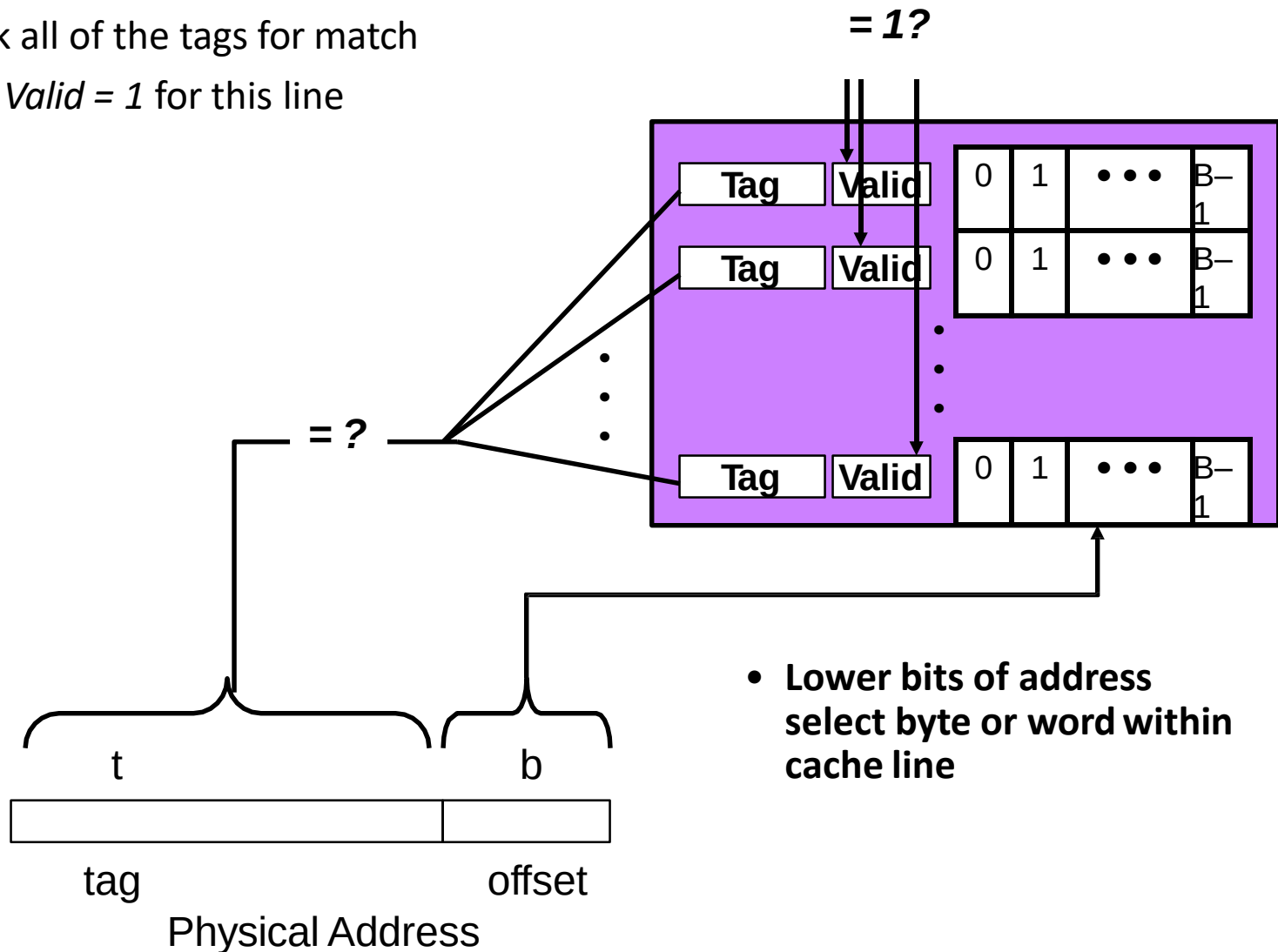
Only practical for small caches in hardware!

Still useful for analysis and simulation

Fully associative cache tag matching

Identifying Line

- Must check all of the tags for match
- Must have *Valid* = 1 for this line



Direct-mapped caches

Simplest Design

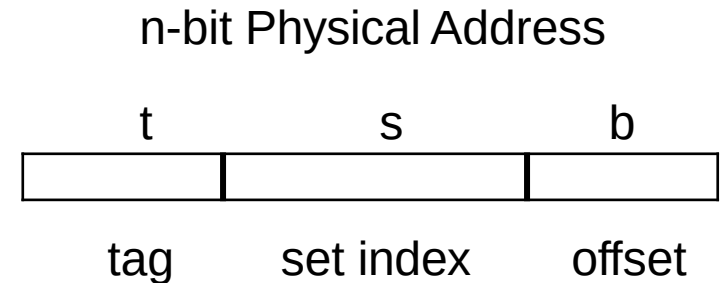
- Each address maps to a **single** cache line

Parameters

- Block = $B = 2^b$ bytes
- Cache size $C = 2^c$
- Cache consists of $S = 2^{c-b} = 2^s$ sets

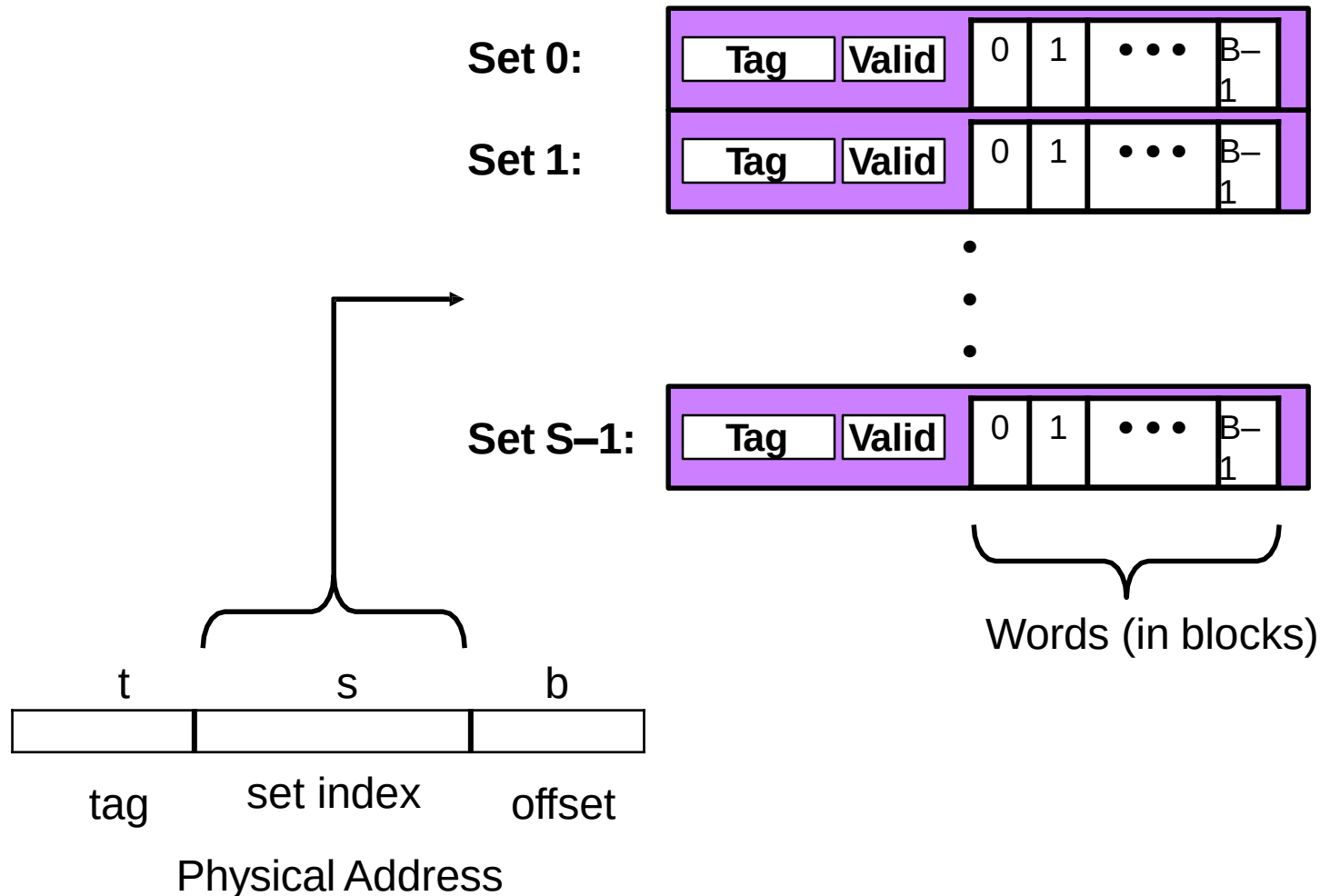
Physical Address

- n bits to reference $N = 2^n$ total bytes
- Partition into fields
 - Offset*: Lower b bits indicate which byte within line
 - Set*: Next s bits indicate how to locate line within cache
 - Tag*: Identifies this line when in cache



Indexing into a direct-mapped cache

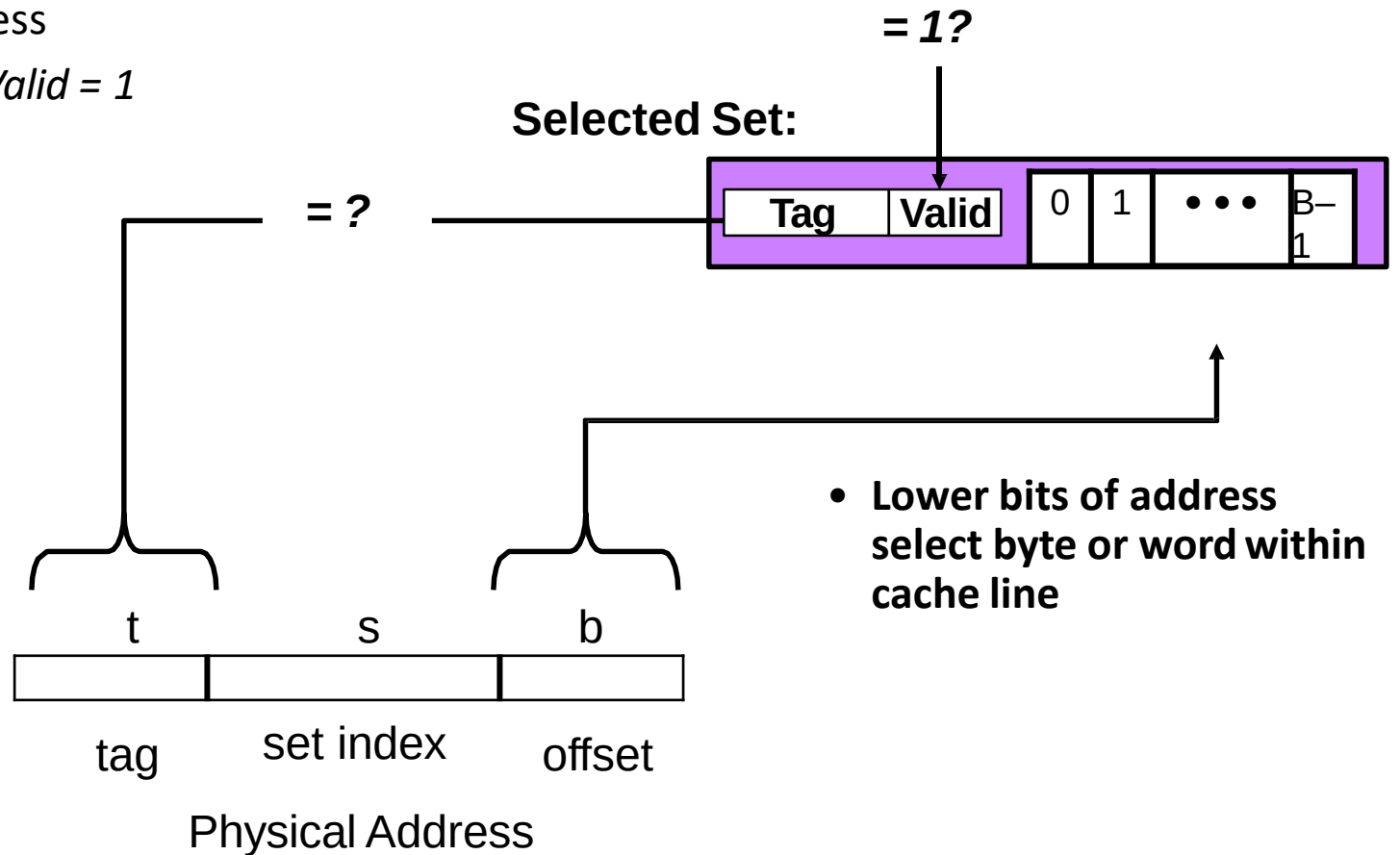
Use set index bits to select cache set



Direct-mapped tag matching

Identifying Line

- Must have tag match high order bits of address
- Must have *Valid* = 1



Ping Pong Effect

Assume a direct-mapped cache with 4 blocks, initially empty, and all blocks are marked as invalid.

Memory Access Order:

0 (0000) 4(0100) 0 4 0 4 0 4

Tag	Block #
-----	---------

Memory Address

0 Miss	4 Miss	0 Miss	4 Miss																																
Tag	Tag	Tag	Tag																																
<table><tr><td>00</td><td>Mem(0)</td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr></table>	00	Mem(0)							<table><tr><td>00</td><td>Mem(0)</td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr></table>	00	Mem(0)							<table><tr><td>01</td><td>Mem(4)</td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr></table>	01	Mem(4)							<table><tr><td>00</td><td>Mem(0)</td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr></table>	00	Mem(0)						
00	Mem(0)																																		
00	Mem(0)																																		
01	Mem(4)																																		
00	Mem(0)																																		

Ping Pong Effect

- **Conflicts:** i.e., thrashing, 8 requests, 8 misses
- Two heavily used lines have same cache index
- Repeatedly evict one to make room for other

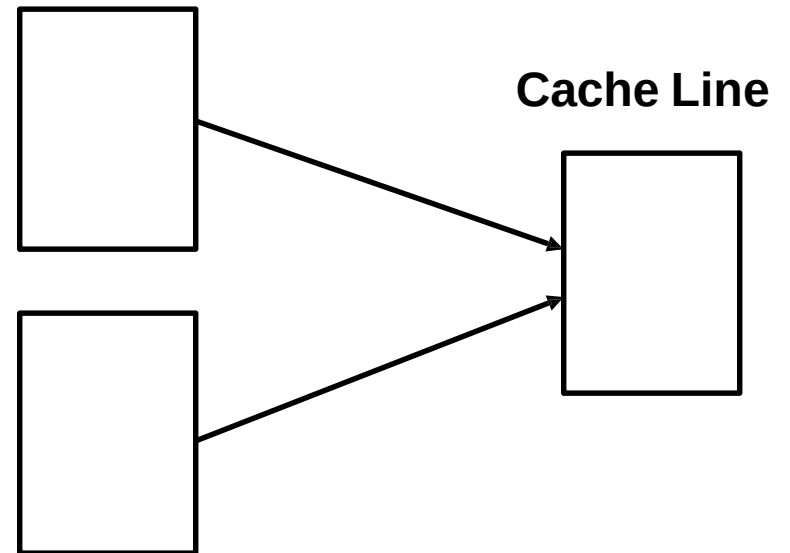
Tradeoffs of direct-mapped caches

Strength

- Minimal control hardware overhead
- Simple design
- Good hit time

Weakness

- Vulnerable to **conflicts** (i.e., thrashing)
- Two heavily used lines have same cache index
- Repeatedly evict one to make room for other



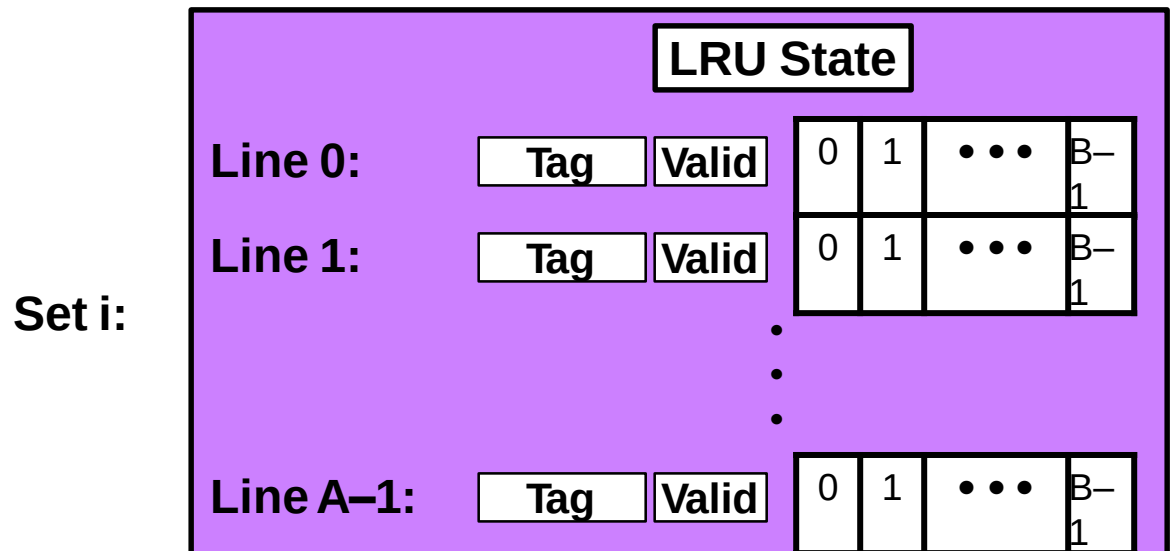
Set-associative cache

Mapping of Memory Lines

- Each address maps to one of A possible locations (a set)
 - $A = 2 - 8$ for L1s, $A = 8 - 32$ for other caches
 - Can map to any entry within its set

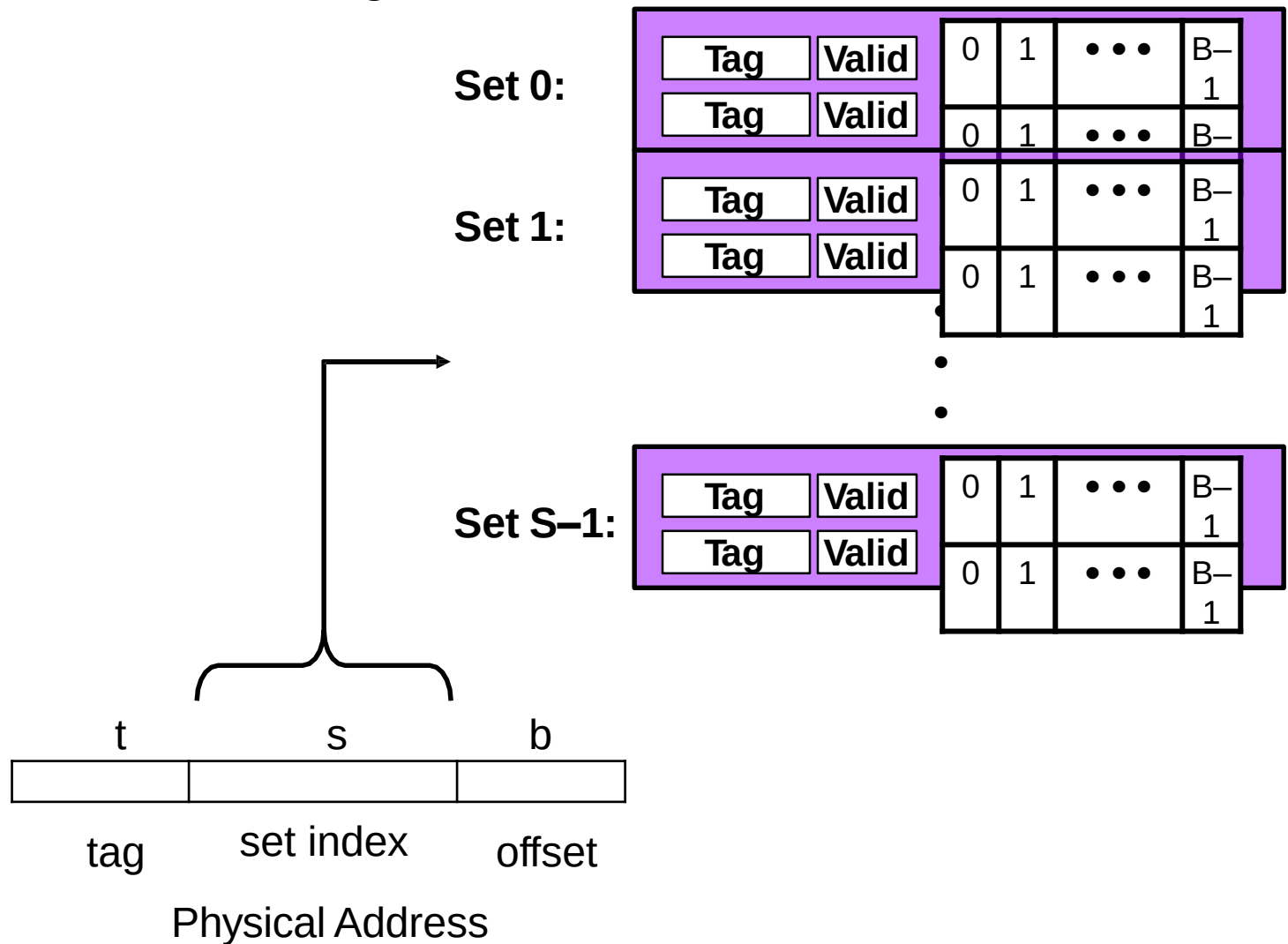
Tradeoffs

- Fewer conflict misses
- Longer access latency
- More complex to implement
- *Forced by virtual memory* (more on this later)



Indexing a 2-way set-associative cache

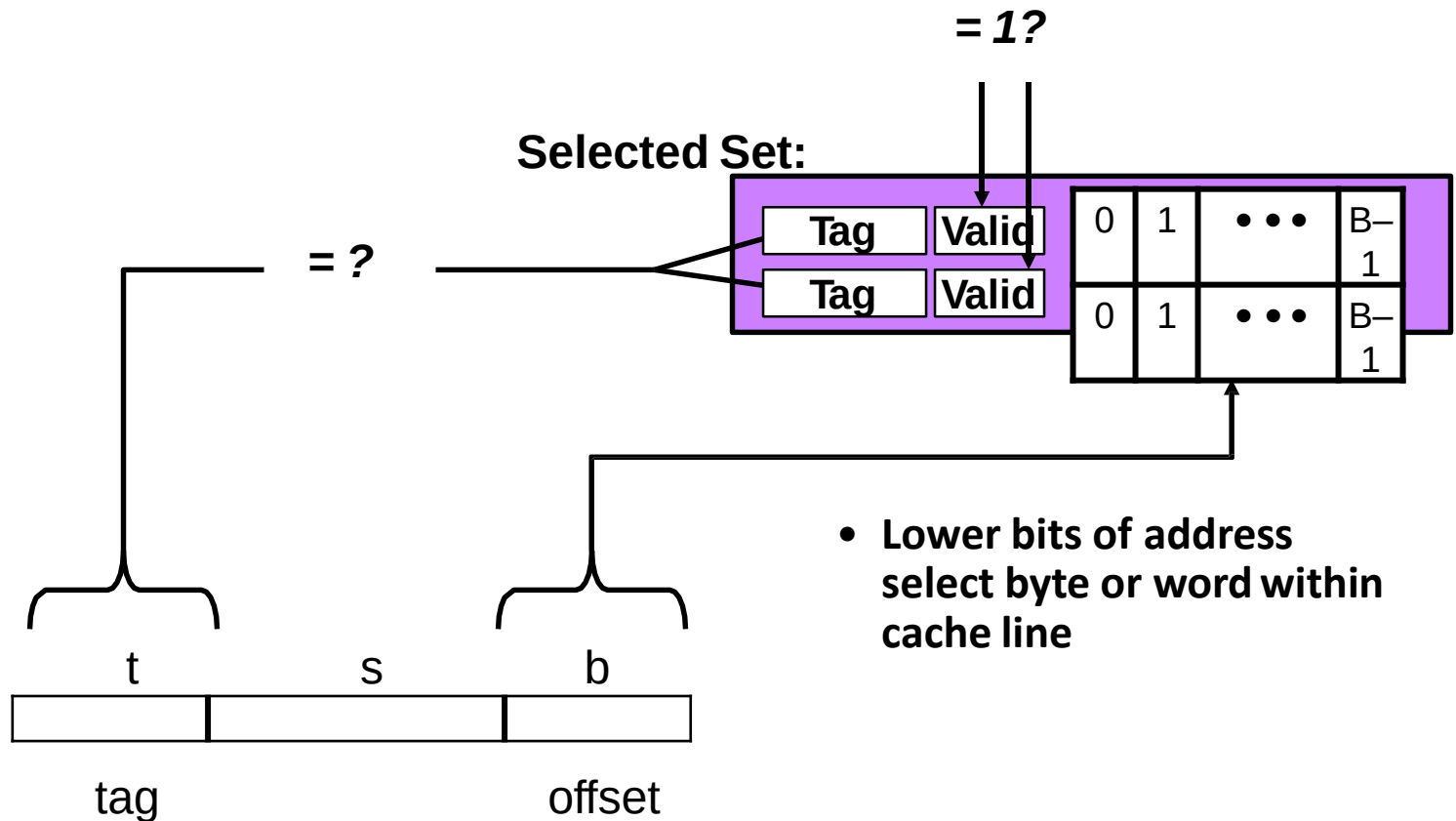
Use middle s bits to select from among $S = 2^s$ sets



Set-associative tag matching

Identifying Line

- Must have one of the tags match high order bits of address
- Must have *Valid* = 1 for this line

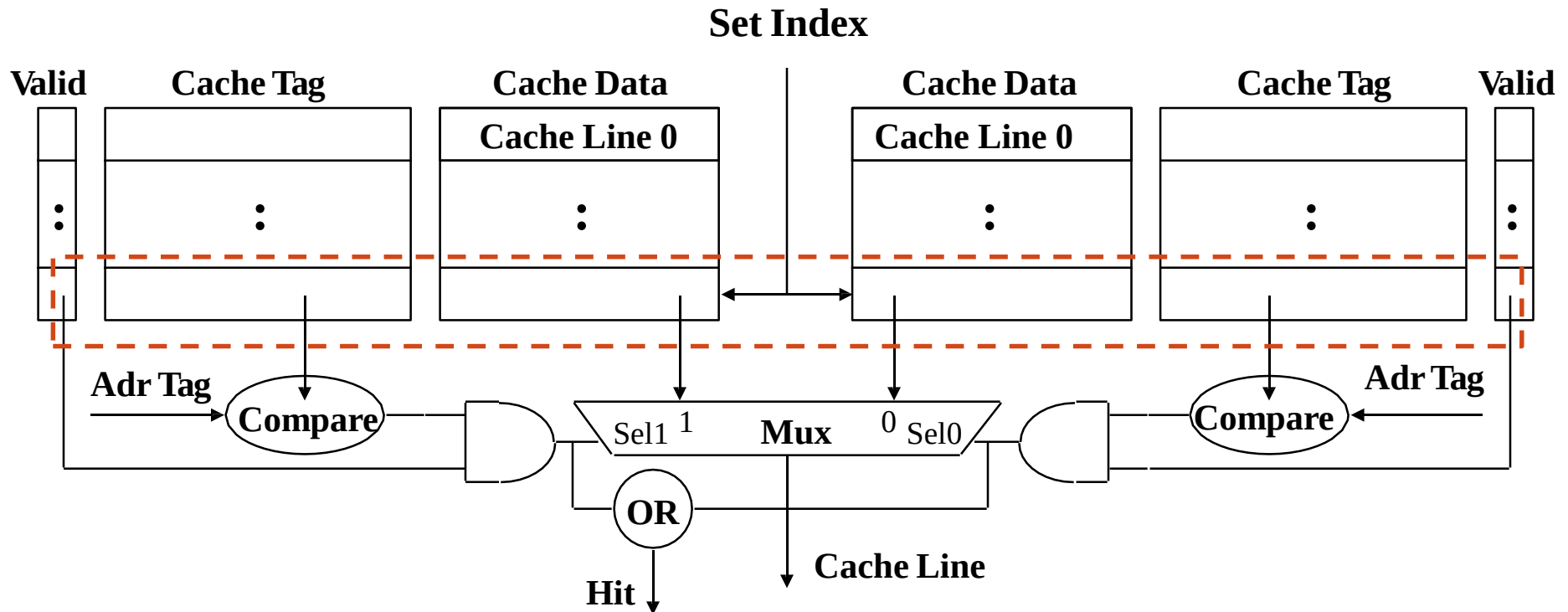


Implementation of 2-way set-associative

Set index selects a set from the cache

The two tags in the set are compared in parallel

Data is selected based on the tag result



Block Identification

How to find the block?

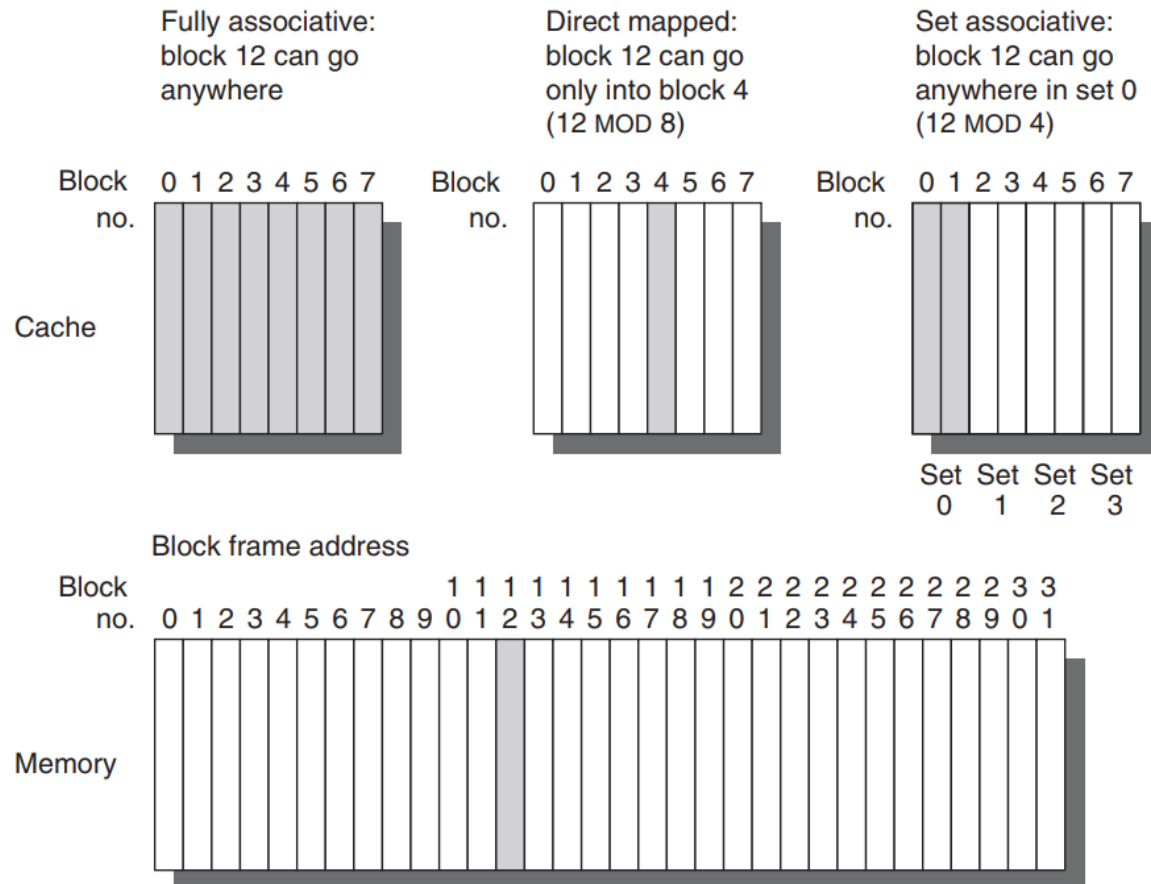
Tag comparisons

Parallel search to speed lookup

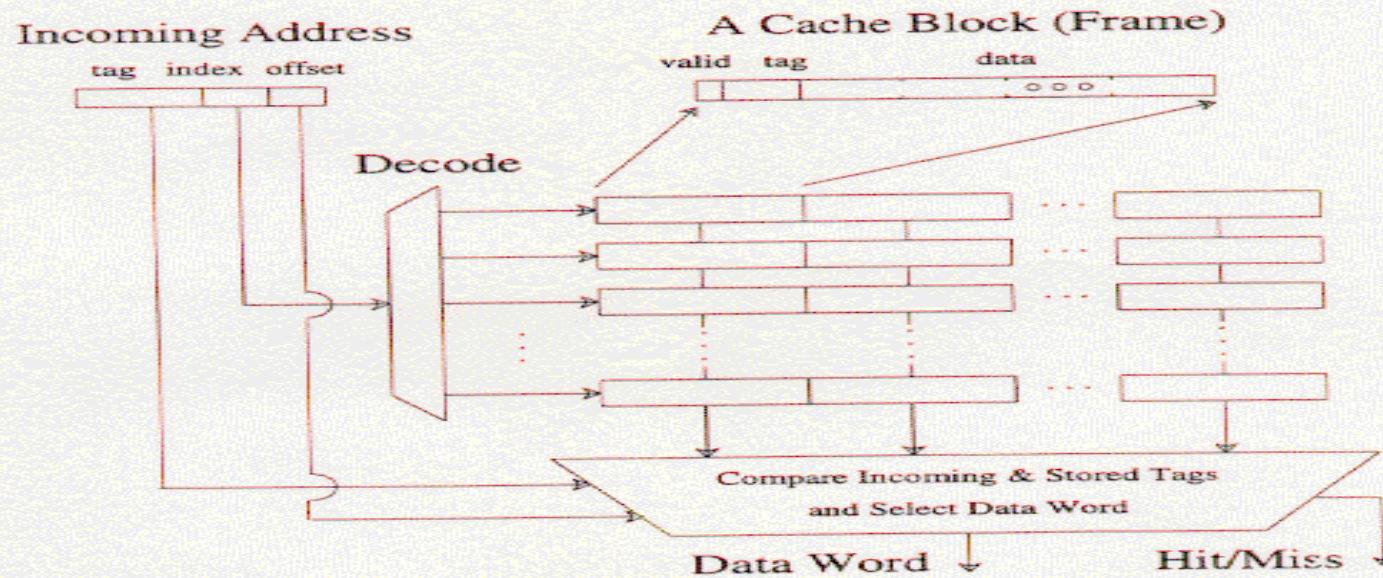
Check valid bit

Example:

Where do we search for block 12?



Example Cache



Block Replacement

Which block to replace on a miss?

Least recently used (LRU)

- Optimize based on temporal locality

- Replace block unused for longest time

- State updates on nonMRU misses

Random

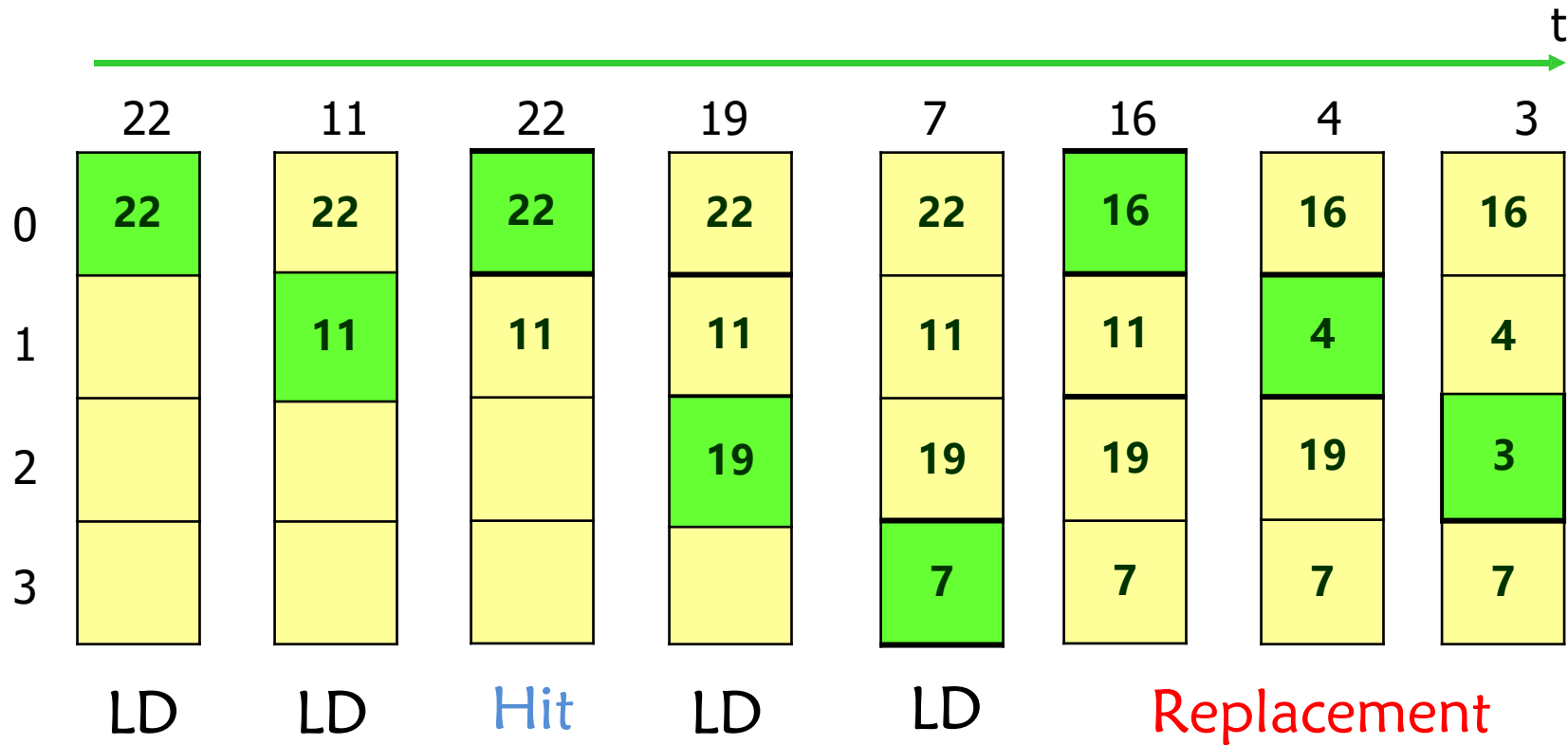
- Select victim at random

- Nearly as good as LRU, and easier

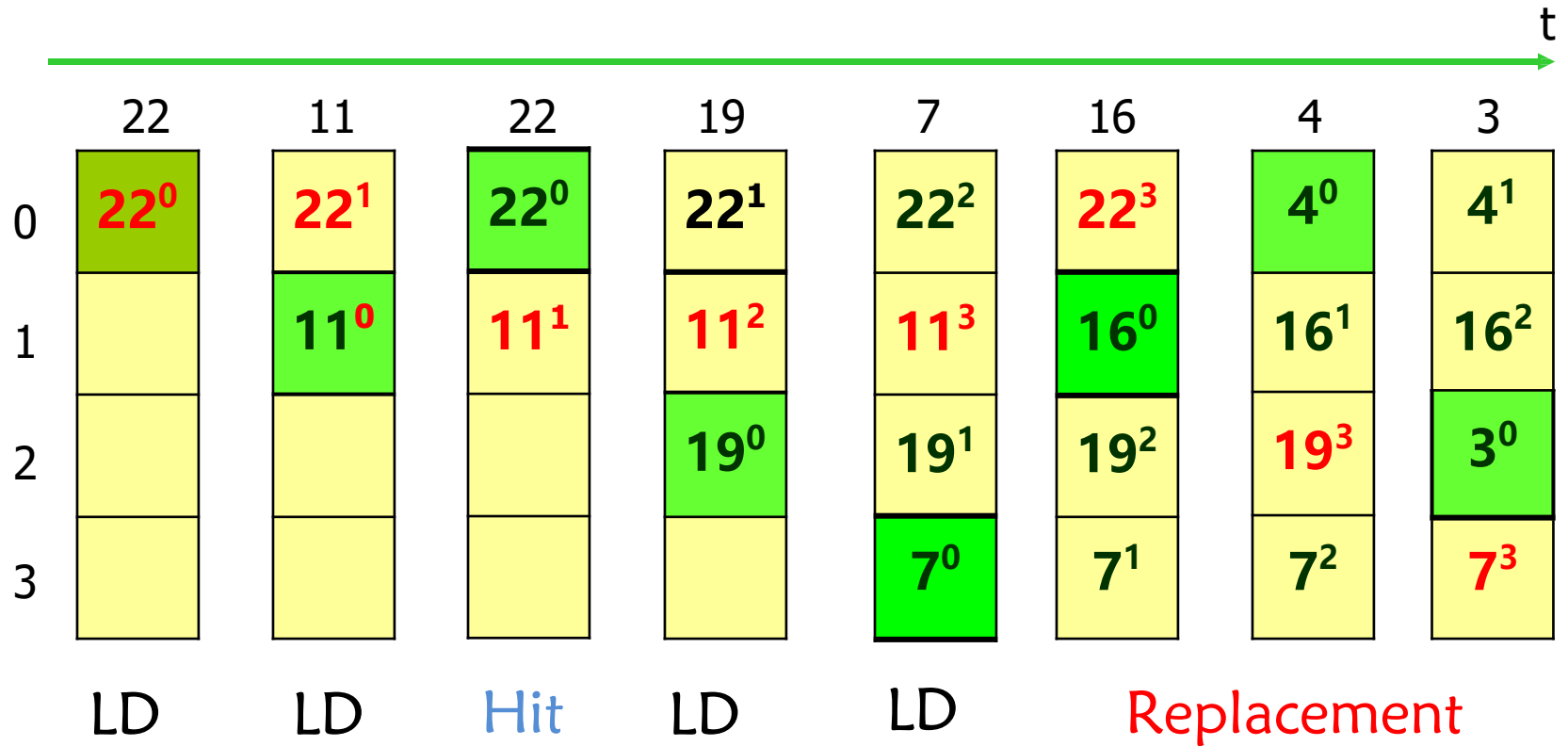
First-in First-out (FIFO)

- Replace block loaded first

Block Replacement —FIFO



Block Replacement —LRU



Write Policies

Writes are harder

Reads done in parallel with tag compare; writes are not

Thus, writes are often slower

(but processor need not wait)

On hits, update memory?

Yes write through (store through)

No write back (store in, copy back)

On misses, allocate cache block?

Yes write-allocate (usually used w/ writeback)

No no-write-allocate (usually used w/ writethrough)

Write Policies, cont.

Write Back

Update memory only on block replacement

Dirty bits used so clean blocks can be replaced without updating memory

Traffic/Reference =

Traffic/Reference =

Less traffic for larger caches

Write Through

Update memory on each write

Write buffers can hide write latency (later)

Keeps memory uptodate (almost)

Traffic/Reference =

Write Policies, cont.**

WriteBack

Update memory only on block replacement

Dirty bits used so clean blocks can be replaced without updating memory

$$\text{Traffic/Reference} = \textit{fractDirty} \times \textit{miss} \times B$$

$$\text{Traffic/Reference} = 1/2 \times 0.05 \times 4 = 0.10$$

Less traffic for larger caches

WriteThrough

Update memory on each write

Write buffers can hide write latency (later)

Keeps memory uptodate (almost)

$$\text{Traffic/Reference} =$$

Write Policies, cont.**

WriteBack

Update memory only on block replacement

Dirty bits used so clean blocks can be replaced without updating memory

$$\text{Traffic/Reference} = \textit{fractDirty} \times \textit{miss} \times B$$

$$\text{Traffic/Reference} = 1/2 \times 0.05 \times 4 = 0.10$$

Less traffic for larger caches

WriteThrough

Update memory on each write

Write buffers can hide write latency (later)

Keeps memory uptodate (almost)

$$\text{Traffic/Reference} = \textit{fractionWrites} = 0.20$$

Traffic independent of cache parameters

Write Policies

Proportion of writes

Stores = 10%, Loads = 26%

Proportion of writes in the overall memory traffic

$10\% / (26\% + 10\% + 100\%) \approx 7\%$

Proportion of writes in the data cache traffic

$10\% / (26\% + 10\%) \approx 28\%$

Cache Type

Unified (mixed)

- Less costly

- Dynamic response

- Handles writes into Istream

Separate Instruction & Data (split, Harvard)

- 2x bandwidth

- Place closer to I and D ports

- Can customize

- Poorman's associativity

- No interlocks on simultaneous requests

Caches should be split if simultaneous instruction and data accesses are frequent (e.g., RISCs)

Cache Type Example

Consider building (a) 16K byte I & D caches, or (b) a 32K byte unified cache.

Let t_{cache} is one cycle, t_{memory} is 10 cycles.

(a) I_{miss} is 5 %, D_{miss} is 6 %, 75 % of references are instruction fetches.

$$t_{avg} =$$

(b) miss ratio is 4 %

$$t_{avg} =$$

Cache Type Example**

Consider building (a) 16K byte I & D caches, or (b) a 32K byte unified cache.

Let t_{cache} is one cycle, t_{memory} is 10 cycles.

(a) I_{miss} is 5 %, D_{miss} is 6 %, 75 % of references are instruction fetches.

$$t_{avg} = (1 + 0.05 \times 10) \times 0.75 \\ + (1 + 0.06 \times 10) \times 0.25 = 1.5$$

(b) miss ratio is 4 %

$$t_{avg} =$$

Cache Type Example**

Consider building (a) 16K byte I & D caches, or (b) a 32K byte unified cache.

Let t_{cache} is one cycle, t_{memory} is 10 cycles.

(a) I_{miss} is 5 %, D_{miss} is 6 %, 75 % of references are instruction fetches.

$$t_{avg} = (1 + 0.05 \times 10) \times 0.75 \\ + (1 + 0.06 \times 10) \times 0.25 = 1.5$$

(b) miss ratio is 4 %

$$t_{avg} = 1 + 0.04 \times 10 = 1.4$$

Cache Type Example**

Consider building (a) 16K byte I & D caches, or (b) a 32K byte unified cache.

Let t_{cache} is one cycle, t_{memory} is 10 cycles.

(a) I_{miss} is 5 %, D_{miss} is 6 %, 75 % of references are instruction fetches.

$$t_{avg} = (1 + 0.05 \times 10) \times 0.75 \\ + (1 + 0.06 \times 10) \times 0.25 = 1.5$$

(b) miss ratio is 4 %

$$t_{avg} = 1 + 0.04 \times 10 = 1.4 \text{ WRONG!}$$

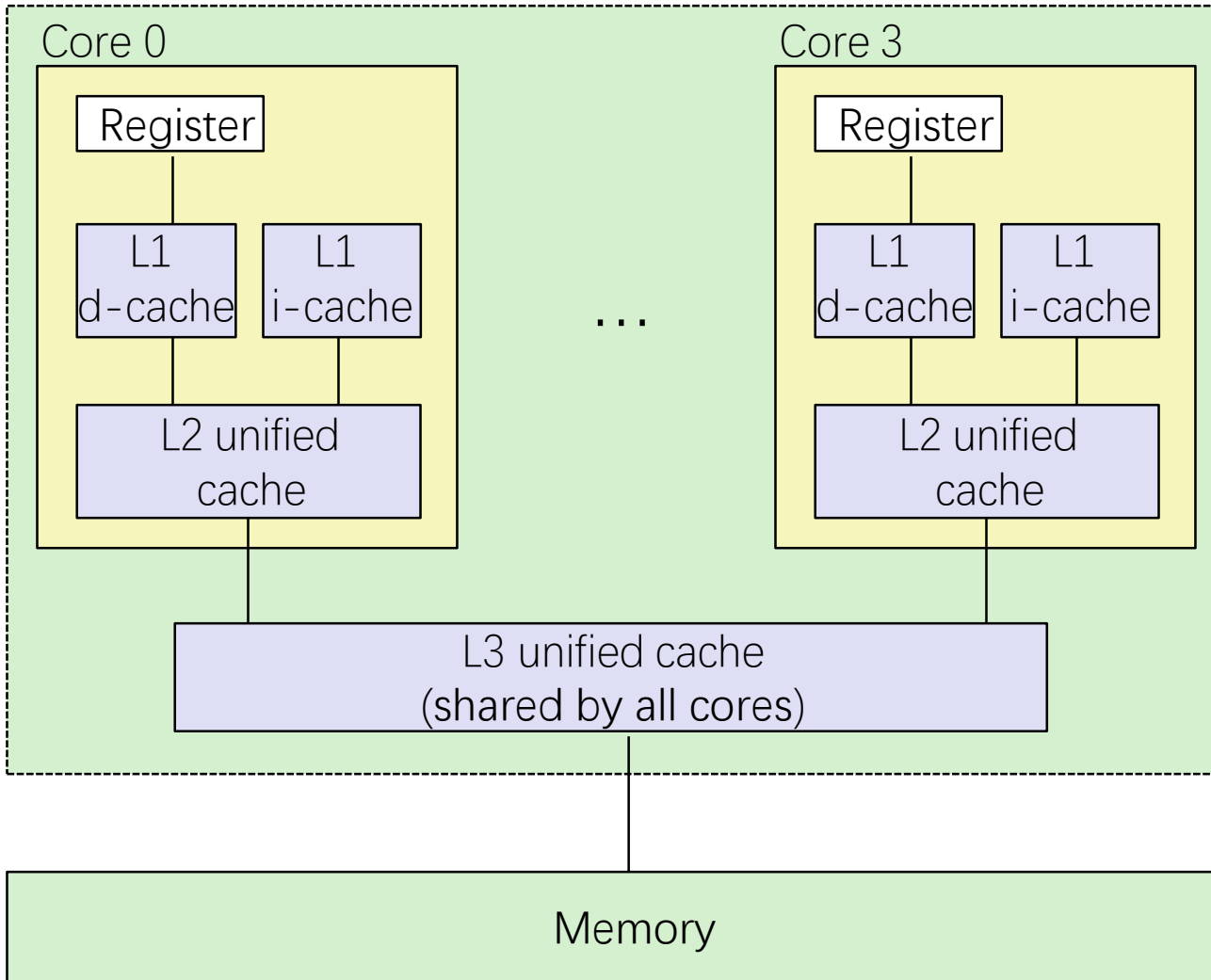
$$t_{avg} = 1.4 + \text{cycles-lost-to-interference}$$

Will cycles-lost-to-interference < 0.1 ?

Not for “RISC” machines!

Cache Type Example**

Intel Core i7



L1 i-cache & d-cache:

32 KB, 8-way,

Access time:

4 cycles

L2 unified cache:

256 KB, 8-way,

Access time:

10 cycles

L3 unified cache:

8 MB, 16-way,

Access time:

40-75 cycles

Block size: 64 bytes

A Miss Classification (3Cs or 4Cs)

Cache misses can be classified as:

Compulsory (a.k.a. cold start)

The first access to a block

Capacity

Misses that occur when a replaced block is re-referenced

Conflict (a.k.a. collision)

Misses that occur because blocks are discarded because of the set-mapping strategy

Coherence (shared-memory multiprocessors)

Misses that occur because blocks are invalidated due to references by other processors

Total Miss Rate to the 3Cs

Compulsory misses are independent of cache size

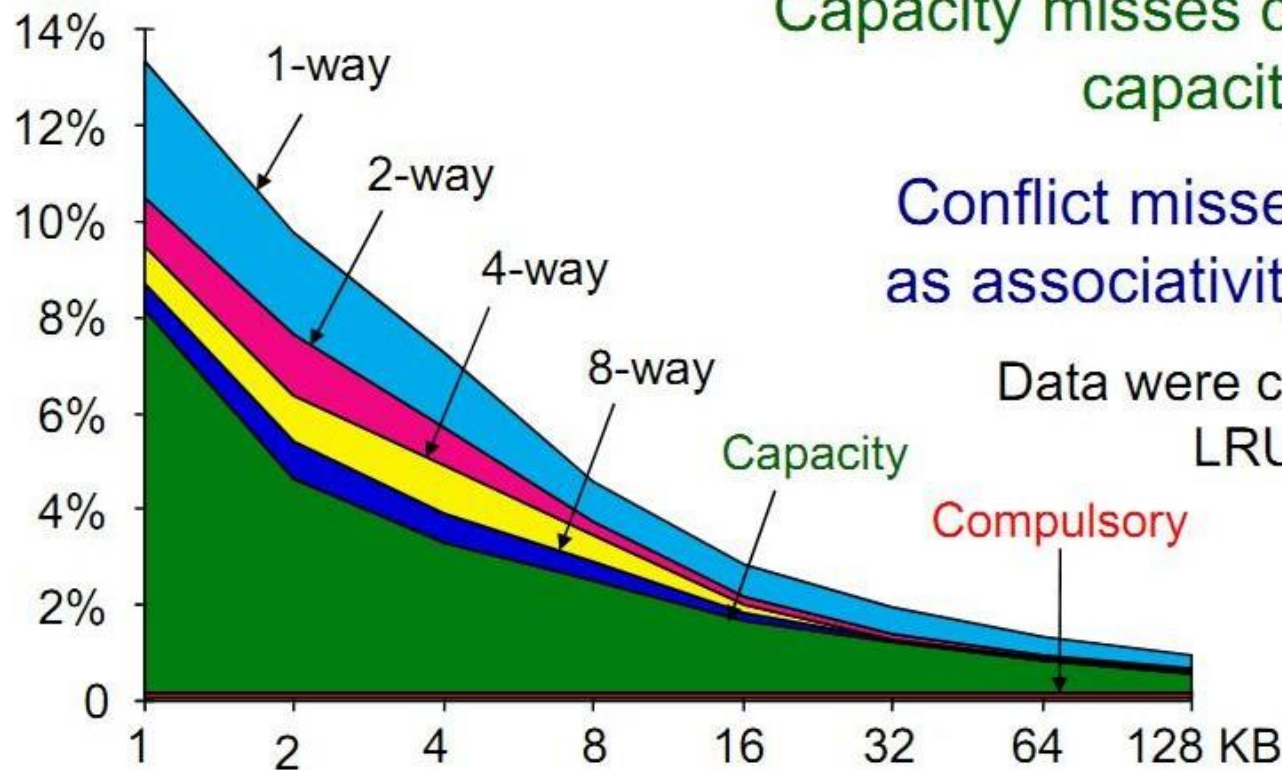
Very small for long-running programs

Capacity misses decrease as capacity increases

Conflict misses decrease as associativity increases

Data were collected using LRU replacement

Miss Rate



Basic Cache Optimizations

Larger block Size (reduce miss rate)

Take advantage of spatial locality, also reduce compulsory misses

Larger Cache (reduce miss rate)

Increase capacity of the cache is the obvious way to reduce miss rate

Higher Associativity (reduce miss rate)

There are two general rules of thumb

Multilevel Caches (reduce miss penalty)

Do both: make the cache faster and bigger

Giving Priority to Read Misses over Writes (reduce miss penalty)

Check the contents of the write buffer on a read miss

Avoiding Address Translation during Indexing of the Cache (reduce hit time)

Virtual cache: virtual addresses for the cache

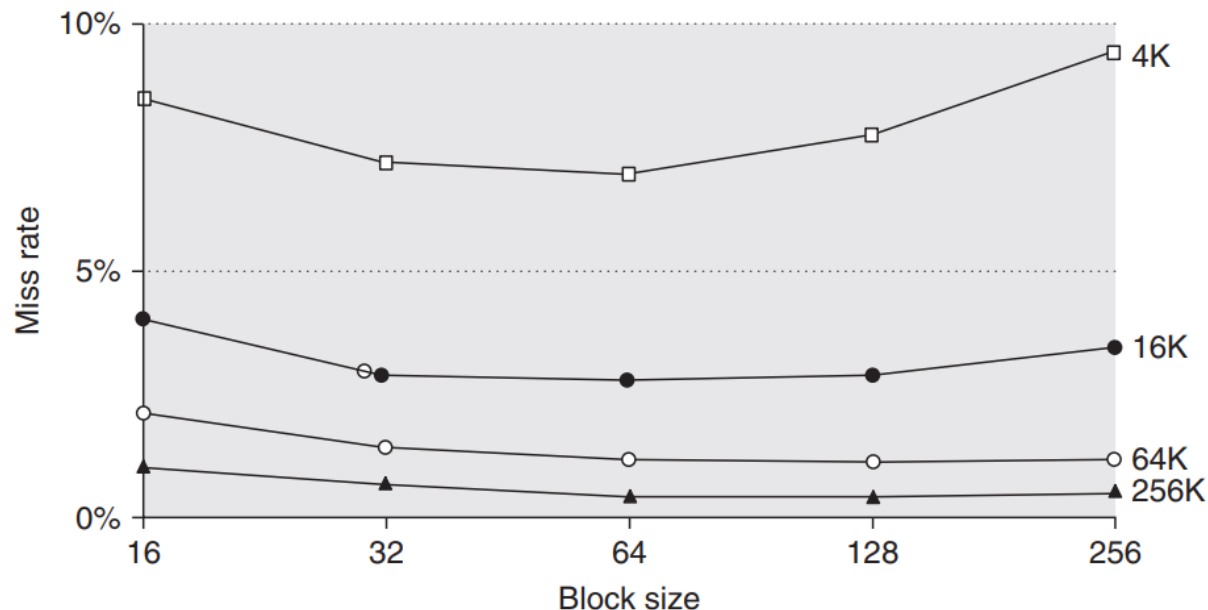
Larger block Size

Larger block Size (reduce miss rate)

Take advantage of spatial locality, also reduce compulsory misses

Miss rate actually goes up if the block size is too large relative to the cache size

The larger the cache, the larger the block size needed to achieve a lower miss rate



Side Effect

Larger blocks, higher miss penalty

Larger Cache

Larger Cache (reduce miss rate)

Increase capacity of the cache is the obvious way to reduce miss rate

Side Effect

longer hit time and higher cost and power

Higher Associativity

Higher Associativity (reduce miss rate)

There are two general rules of thumb

Two general rules of thumb

- 1. eight-way set associative is for practical purposes as effective in reducing misses for these sized caches as fully associative*
- 2. 2:1 cache rule of thumb: a directmapped cache of size N has about the same miss rate as a two-way set associative cache of size $N/2$*

Side Effect

Higher associativity, higher hit time

Multilevel Caches

Faster cache? Bigger cache?

Multilevel Caches (reduce miss penalty)

Do both: make the cache faster and bigger

L1 Cache: small enough to match the clock cycle time of the fast processor

L2 Cache: large enough to lessen the effective miss penalty

Average (effective) access time is better

$$\begin{aligned} t_{avg} &= t_{cache-L1} + miss\ ratio_{L1} \times miss\ penalty_{L1} \\ &= t_{cache-L1} + miss\ ratio_{L1} \times (t_{cache-L2} + miss\ ratio_{L2} \times miss\ penalty_{L2}) \end{aligned}$$

Multilevel Caches

Local miss rate

= the number of misses in a cache / the total number of accesses to this cache

Global miss rate

= the number of misses in the cache / the total number of accesses generated by the processor

Global miss rate

= $\text{miss ratio}_{L1} \times \text{miss ratio}_{L2}$

The global miss rate is the more useful measure for multilevel caches:

The fraction of the memory accesses that leave the processor go all the way to memory

Average memory stalls per instruction

= Misses per instruction_{L1} × Hit time_{L2} + Misses per instruction_{L2} × Miss penalty_{L2}

Giving Priority to Read Misses over Writes

Giving Priority to Read Misses over Writes (reduce miss penalty)

Check the contents of the write buffer on a read miss

The complexities of a write buffer

With a write-through cache the most important improvement is a write buffer of the proper size.

Write buffers might hold the updated value of a location needed on a read miss

The solutions:

*1. The read miss waits until the write buffer is empty
(Higher the miss penalty)*

*2. Check the contents of the write buffer on a read miss (Add the hardware)
i.e. Giving reads priority over writes*

Avoiding Address Translation during Indexing of the Cache

Avoiding Address Translation during Indexing of the Cache (reduce hit time)

Virtual cache: virtual addresses for the cache

why doesn't everyone build virtually addressed caches?

1. One reason is protection

Solution: Copying the protection information from the TLB on a miss, add a field to hold it, and check it on every access to the virtually addressed cache

2. Another reason is that a process is switched, the virtual addresses refer to different physical addresses, requiring the cache to be flushed

Solution: Increasing a process-identifier tag (PID). If the operating system assigns these tags to processes, it only need flush the cache when a PID is recycled

3. A third reason is synonyms or aliases (two different virtual addresses for the same physical address)

Solution: Antialiasing and Page coloring

Acknowledgements

These slides contain material developed and copyright by:

- Sarita Adve (UIUC)
- Nathan Beckmann (CMU)
- David Patterson (UC Berkeley)
- Qianni Deng (SJTU)
- Changsheng Xie (HUST)